Neural Networks 206 (2027) 109765

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

CODER: Data-efficient activation of mathematical reasoning via difficulty-aware core-set

Lei Wang, GuoAo Liu *, XiaoYong Zhao

Beijing Information Science & Technology University, 102206, Beijing, China

ARTICLE INFO

Keywords:
Large language models
Mathematical reasoning
Difficulty structure
Low computing power

ABSTRACT

Large language models (LLMs) demonstrate remarkable performance in mathematical reasoning tasks, yet their fine-tuning process typically relies on substantial volumes of high-quality annotated data. Under conditions of limited computational resources and sparse data, they are highly susceptible to overfitting and catastrophic forgetting. Inspired by the classic textbooks used in human education from primary to secondary school, which embody a natural difficulty progression, this work suggests that constructing hierarchical data covering key knowledge nodes can effectively facilitate the activation of a model's logical reasoning capabilities with minimal sample cost. This paper proposes CODER, a data-efficient fine-tuning framework based on difficulty-aware set construction and collaborative regularisation. First, we constructed Math-Core-700 from a candidate pool spanning the full primary-to-university curriculum — textbook exercises, provincial mock examinations, publicly released competition problems and introductory university mathematics such as calculus. This dataset features a complete difficulty gradient spanning fundamental operations to advanced abstract concepts. By purposefully incorporating final-stage examination questions and competition proof problems, we facilitated the model's transition from simple pattern mimicry to enhanced logical reasoning. Secondly, we devise a collaborative fine-tuning strategy that leverages DoRA to decouple the magnitude and direction of parameter updates, while integrating NEFTune to inject noise into the feature space to promote manifold distribution smoothing. Empirical results provide supportive evidence that this approach achieves 90.6%(mean) and 82.9% (pass@16) accuracy on GSM8K and AMC 23 respectively using only 700 samples, achieving competitive or superior performance compared to standard LoRA baselines trained on the MATH dataset (7.5k samples) and TAL-SCQ5K dataset (5k samples). This finding suggests that during downstream fine-tuning, data filtering strategies based on a knowledge-grounded Difficulty-Aware Core-Set can potentially mitigate the diminishing returns associated with data scale. Consequently, they enable the effective activation of model capabilities without relying on extensive labelled data, thereby exploring new pathways for popularising high-performance large model training in resource-constrained environments.

1. Introduction

In recent years, large language models (LLMs) based on the Transformer architecture have achieved landmark progress in the field of natural language processing. From GPT-4 (OpenAI, 2023) to open-source community models such as Llama-3 (Dubey et al., 2024) and Qwen-3 (Yang et al., 2025), these systems have demonstrated remarkable "emergent" capabilities in text generation, code writing, and multi-turn dialog. However, despite their excellence in general-purpose tasks, LLMs still face significant challenges when handling mathematical reasoning tasks requiring rigorous logical deduction. Mathematical reasoning demands multi-step causal inference capabilities, whereas autoregressive models based on probabilistic prediction are highly prone to logical hallucinations or computational errors (Dziri et al., 2023).

To enhance the mathematical capabilities of models, the mainstream academic paradigm follows scaling laws (Kaplan et al., 2020), whereby performance gains are achieved through the continuous increase in model parameters (e.g., hundreds of billions of parameters) and training data volumes (e.g., mathematical corpora comprising trillions of tokens). Pioneering works such as Google's Minerva (Lewkowycz et al., 2022) and DeepSeek-Math (Shao et al., 2024) have demonstrated the efficacy of this approach. However, this resource-intensive paradigm imposes substantial barriers for academia and smaller research institutions. Full-scale fine-tuning not only demands costly computational

* Corresponding author.
E-mail address: 13395474267@163.com (G. Liu).

https://doi.org/10.1016/j.neunet.2026.109765
Received 2 February 2026; Received in revised form 30 September 2026; Accepted 11 October 2026
Available online 19 October 2026
0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

clusters but also, in domains with sparse data, frequently leads to overfitting on training samples and catastrophic forgetting of general capabilities (Kirkpatrick et al., 2017).

Addressing these challenges, existing solutions primarily focus on two dimensions: parameter-efficient fine-tuning (PEFT) and data engineering. Nevertheless, significant limitations persist:

At the algorithmic level, while Hu et al.'s LoRA (Low-Rank Adaptation) (Hu et al., 2022) substantially reduces memory overhead, its randomly initialized low-rank matrices often suffer from insufficient learning capacity when handling complex logical tasks due to constraints imposed by parameter update coupling. Liu et al.'s 2024 DoRA (Weight-Decomposed LoRA) (Liu et al., 2024) enhances performance by decoupling weight magnitude and direction, yet its convergence stability under ultra-small samples remains to be validated. Furthermore, addressing overfitting in few-shot fine-tuning, Jain et al.'s NEFTune (Jain et al., 2023) achieves regularization through noise injection, yet its application mechanism for demanding logical-mathematical tasks remains underresearched.

At the data level, Zhou et al.'s LIMA (Less Is More for Alignment) (Zhou et al., 2023) demonstrates that a small volume of high-quality data suffices for model capability alignment. However, existing open-source mathematics datasets (such as GSM8K and MATH) largely lack fine-grained difficulty grading, and the quality of their reasoning processes (Chain-of-Thought, CoT) (Wei et al., 2022) varies considerably. Directly fine-tuning on such datasets, which carry cognitive gaps, makes it difficult for a model to construct a complete cognitive chain from basic operations to abstract reasoning within a limited sample budget. This work therefore makes the following contributions:

- Math-Core-700: a 700-sample CoT set with a difficulty pyramid spanning elementary to competition and university mathematics.
- DoRA magnitude-direction decoupling and NEFTune noise form a geometric-stochastic synergy for sparse-data generalization.
- 700 samples: CODER matches baselines trained on 7,500 MATH and 5000 TAL-SCQ5K, generalising OOD to AMC 23 and Hard-CN.
- CODER exceeds the 7,500-example MATH-LoRA baseline using about one tenth of its optimisation steps, within a 24GB budget.

2. Relevant work

The Paradigm Shift in Mathematical Reasoning: From Chain of Thought to Process Reward Large Language Models (LLMs) have undergone a paradigm shift in mathematical reasoning tasks, transitioning from outcome-oriented to process-oriented approaches. Early Chain of Thought (CoT) prompting methods (Kojima et al., 2022; Wei et al., 2022) effectively activated models' logical potential by explicitly generating reasoning steps. Subsequently, research focus gradually shifted toward optimizing reasoning pathways using large-scale synthetic data and reinforcement learning. DeepSeek-Math (Shao et al., 2024) and Qwen2.5-Math (Yang et al., 2024) demonstrated the feasibility of constructing powerful foundational models through multi-stage pre-training and supervised fine-tuning (SFT). Recent research has further explored the depth and breadth of reasoning. DeepSeek-R1 (Guo et al., 2025) introduced an evolution mechanism for reasoning based on large-scale reinforcement learning, demonstrating that incentivising models to engage in self-exploration through lengthy thought chains can yield problem-solving strategies surpassing human expectations. Concurrently, rStar (Guan et al., 2025) proposed a reasoning enhancement framework tailored for small language models (SLMs), significantly boosting the performance of 7B-scale models through multi-step verification and self-improvement.

However, these state-of-the-art approaches primarily adhere to scaling laws, relying on trillions of tokens of training data or costly process reward model annotation during pre-training (Lightman et al., 2023; Wang et al., 2024a). They also demand substantial data during fine-tuning, making this high-cost training paradigm difficult to replicate in computationally constrained scenarios. In contrast, this study explores an efficient non-RL pathway by proposing the construction of a difficulty-aware core dataset (Math-Core-700). This enables the activation of deep reasoning capabilities in base models solely through high-quality SFT, without relying on external reward models.

Core-set Selection and Synthetic Data: As the Data-Centric AI paradigm gains traction, academia is re-evaluating the diminishing returns of data scale. The LIMA hypothesis (Zhou et al., 2023) established the theoretical foundation for "less is more". Within mathematics, MathScale (Tang et al., 2024) and OpenMathInstruct-2 (Toshniwal et al., 2024) validated the role of large-scale synthetic data in enhancing generalization. Recent approaches such as Evol-Instruct (Luo et al., 2023) and Magpie (Xu et al., 2024) have generated substantial high-difficulty samples by automatically increasing instruction complexity. Research trends in 2025 increasingly focus on data information density. Less-is-More-for-Math (Ye et al., 2025) demonstrates that removing 90% of redundant data via gradient-based importance filtering preserves model performance.

Nevertheless, existing data filtering strategies predominantly rely on statistical metrics (such as perplexity or loss values), neglecting the intrinsic difficulty structure of mathematical knowledge systems. Simple random sampling or difficulty-based filtering readily leads to knowledge gaps. In this work, we propose a difficulty-pyramid-based data construction strategy, specifically incorporating final-stage university entrance examination questions and competition problems as logical bridges between foundational operations and abstract reasoning. This achieves coherent knowledge activation within small sample sizes.

Parameter Efficient Fine-Tuning and Regularization Mechanisms: Parameter Efficient Fine-Tuning (PEFT) aims to reduce the memory overhead of full-scale fine-tuning. LoRA (Hu et al., 2022) and its variants have become industry standards. To overcome LoRA's parameter coupling limitations in complex inference tasks, DoRA (Liu et al., 2024) introduced a weight decomposition mechanism in 2024, significantly enhancing learning capacity by decoupling magnitude and direction. Concurrently, PiSSA (Meng et al., 2024) adopts a spectral analysis approach, utilizing SVD to initialize principal components for accelerated convergence. Recent developments such as LoRA-Pro (Wang et al., 2024b) and FourierFT (Gao et al., 2024) further explore improvements at the frequency domain and optimiser levels, seeking a more optimal balance between parameter efficiency and expressive capability.

Nevertheless, overfitting remains a core bottleneck in low-resource fine-tuning. Traditional regularization methods (e.g., Dropout) often compromise the semantic coherence of pre-trained models. NEFTune (Jain et al., 2023) achieves implicit data augmentation by injecting noise into embedding layers, yet its efficacy in tasks with strong logical constraints remains under-validated. This research innovatively constructs a synergistic regularization framework combining DoRA (Liu et al., 2024) and NEFTune (Jain et al., 2023). It leverages DoRA's geometric decoupling properties to enhance feature learning precision while utilizing NEFTune (Jain et al., 2023)'s manifold smoothing characteristics to suppress overfitting on small samples. This achieves efficient inference activation on large language models.

3. Methodology

This chapter introduces the CODER method and details its comprehensive implementation pathway. As illustrated in Fig. 1, the framework comprises two components: a core dataset constructed based on difficulty gradients (Math-Core-700) and a collaborative fine-tuning strategy grounded in decoupling and noise.

3.1. Difficulty-aware core-set construction

Within the data-centric AI paradigm, data quality and information density are pivotal in determining model performance upper bounds. Unlike existing approaches that rely on extensive synthetic data, this

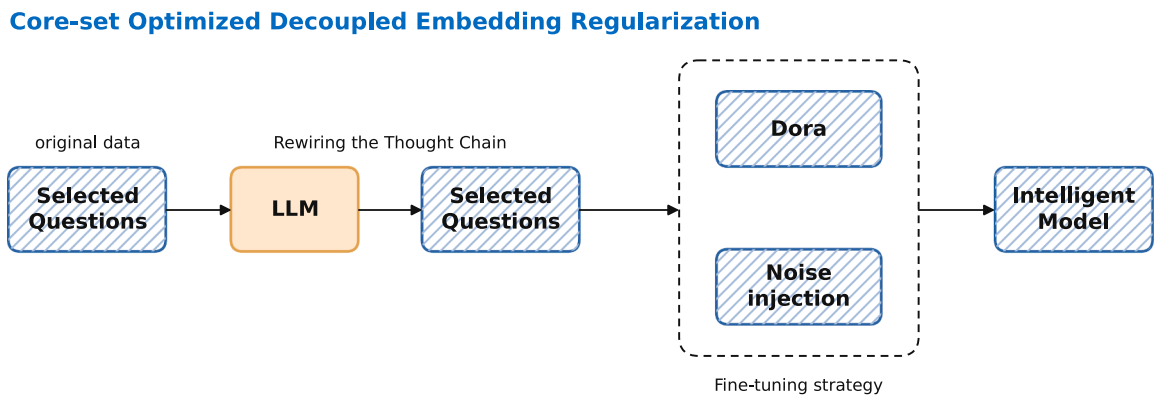


Fig. 1. Overview of the CODER framework. An LLM first reconstructs the reasoning chains of curated problems to yield a high-quality core set; a collaborative fine-tuning strategy combining DoRA and noise injection is then applied to the base model.

work explores the feasibility of a minimal core-set for mathematical reasoning. Throughout this paper, we use "core-set" loosely to denote an empirical representative subset constructed via difficulty stratification, rather than a theoretically optimal core-set in the algorithmic sense (e.g., gradient-matching or k-center). By designing rigorous difficulty gradients and reconstructing high-quality chain-of-thought reasoning, we investigate how difficulty structure and data quality influence the reasoning capabilities of large language models.

3.1.1. Knowledge stratification and difficulty pyramid design

Rather than relying on sampling that ignores difficulty structure or on complex topological filtering, we construct our training subset as a difficulty-stratified empirical curriculum, illustrated in Fig. 2. The tiers do not aim to bridge abstract cognitive gaps; they are combined so as to guide the base model along a progressive learning trajectory. The candidate pool spans the full curriculum rather than a single grade band or source type—post-lesson exercises from primary-, lower-secondary- and senior-secondary-level textbooks, challenging junior-secondary problems, senior-secondary supplementary practice, recent provincial mock examination papers for the National College Entrance Examination, final-stage examination problems, publicly released competition problems, and introductory university mathematics such as calculus.

To render the selection criterion reproducible, we operationalize it as a strictly enforced 30%–50%–20% distribution over the Easy, Medium and Hard tiers, which yields Math-Core-700. Tier assignment was carried out jointly by two university mathematics teachers, who judged each candidate by the conceptual prerequisites it assumes, the depth of the reasoning chain it demands, and the mathematical domain it belongs to. Within each tier, items were drawn so as to maximize coverage of a knowledge-point taxonomy derived from the national curriculum standard while preserving source-type diversity, rather than uniformly at random. The candidate pool comprised approximately 5000 problems; integrity screening — de-duplication, verification of final answers, and rejection of items whose reasoning chains failed our formatting and consistency checks — was applied before the draw. The resulting set covers 240 of the 259 knowledge points (92.7%); candidates on which the two judgments diverged were revisited, and their tier settled by discussion before admission.

1. **Foundation Layer**: Serving as the broad base of the pyramid, this tier comprises foundational post-lesson exercises from primary and lower secondary education, constituting approximately 30% of the dataset. The core objective of this tier is to facilitate basic symbolic alignment within the model. While the foundation model possesses basic computational capabilities, it may exhibit format drift or hallucinations when processing multi-step arithmetic operations and basic application problems. By introducing these structurally standardized and logically straightforward samples, we aim to encourage consistent output formatting (e.g., LaTeX standardization) and reinforce fundamental operational rules, which may help establish a stable basis for subsequent complex reasoning.
2. **Core Layer**: Building upon this foundation, we construct the core logical tier, which constitutes 50% of the dataset. Data sources include challenging junior secondary school problems, high-quality senior secondary supplementary exercises, and recent provincial mock examination papers for the National College Entrance Examination. This tier covers core examination topics such as functions, geometry, and sequences. Compared to foundational problems, mock examination questions typically involve greater complexity, longer reasoning chains, and more tightly coupled logical dependencies. Training on such problems encourages the model to move beyond surface-level pattern matching and develop more structured reasoning pathways within the representation space, thereby supporting the acquisition of standard problem-solving strategies for multi-step mathematical challenges.
3. **Apex Layer**: To further assess the model's reasoning boundaries and enhance its out-of-distribution (OOD) generalization capabilities, we introduced approximately 20% competitive proof problems and introductory university mathematics questions at the pyramid's apex. Competitive proofs demand rigorous logical deduction and creative thinking, encouraging the model to engage in formal reasoning rather than relying on numerical heuristics; introductory university problems (e.g., calculus) introduce higher-dimensional abstract concepts. This tier is intended to promote the model's ability to generalize underlying logical principles to novel and heterogeneous problems, potentially reducing the risk of overfitting to school-level problem templates.

The question of where the alignment between a problem and the model's knowledge is to be established also arises one step earlier, at the level of data selection. In a companion line of work—a case study on medical instruction-tuning—(Zhong et al., 2026) score each candidate instance by its *context–memory knowledge alignment* and its *intra-memory knowledge consistency*, retaining instances whose contextual knowledge agrees with the model's pretrained memory and discarding those in conflict, so that domain knowledge can be injected without eroding prior competence or inducing hallucination. Their criterion and ours measure different properties, and the difference is worth stating precisely. KDS scores an instance by its *agreement with what the model already knows*; our tier rubric places an instance by its *distance from what the model can currently do*, that is, by the depth of the reasoning chain it demands. The two are therefore orthogonal rather than opposed: difficulty is not a proxy for knowledge conflict, and a consistency filter is not a substitute for a difficulty-stratified curriculum. What the two share is a concern with *quality rather than quantity* of supervision—the motivation behind KDS is that poorly matched data damages prior abilities, and the same concern

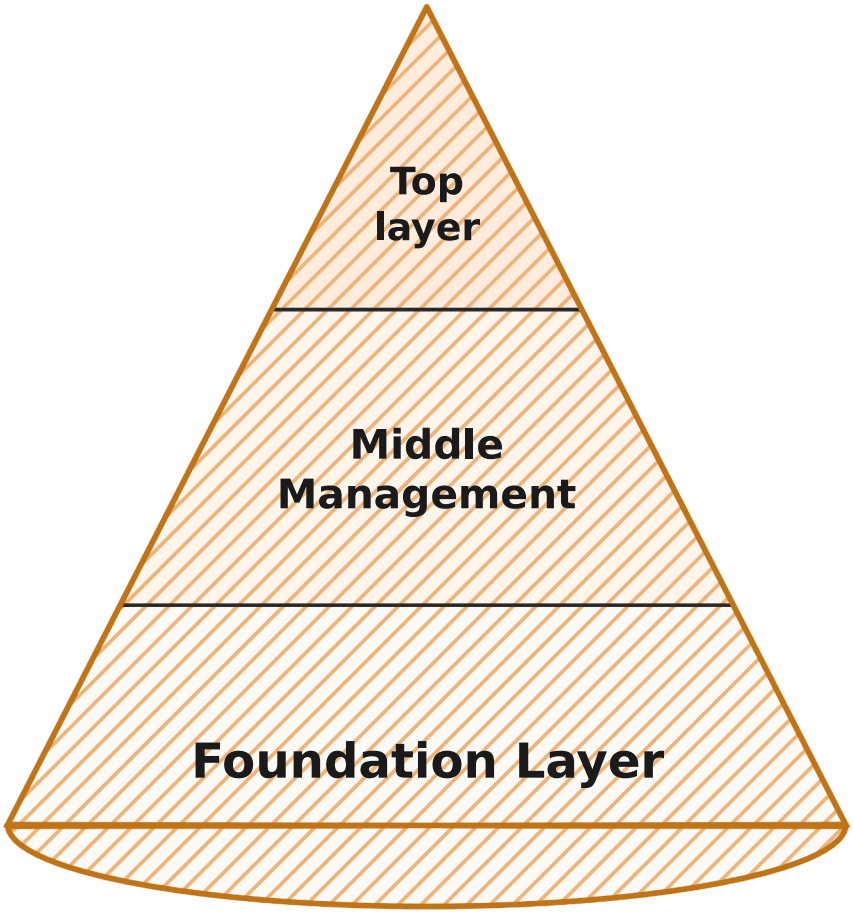


Fig. 2. Hierarchical structure of the difficulty pyramid.

underlies our assistant-only objective and the catastrophic-forgetting analysis in Section 4.2—and this is where they can be combined. The alignment and consistency scores provide a model-aware gate that can be applied *on top of* an expert-assigned level rubric, rejecting instances whose problem statements are ambiguous, internally inconsistent or irreconcilable with the backbone's knowledge while leaving the difficulty composition of the set intact. Our knowledge-point taxonomy already supplies the item-level knowledge attribution such a gate requires, which is why we regard the adaptation as tractable, and we identify this joint, expert-plus-model-aware procedure as a direction for future work.

3.1.2. Automated high-density thought chain reconstruction

The original mathematical datasets typically provide only the problem x and final answer y, or contain only brief intermediate steps. This outcome-oriented supervision signal makes it difficult for base models to capture the complex mapping between problems and answers. Particularly when handling the final examination questions and competition problems introduced in this study, the absence of explicit reasoning paths may lead to logical leaps during model fine-tuning. To address this issue, we design an automated data augmentation pipeline based on a teacher model (GPT-5), which aims to reconstruct sparse supervisory signals into high-density reasoning chains, with the goal of maximizing the information content per sample.

From the perspective of probabilistic generation, traditional supervised fine-tuning (SFT) aims to maximize the conditional probability $P(y \mid x)$. However, for complex logical reasoning tasks, directly modeling this probability distribution presents substantial optimization challenges. We introduce a latent variable z to represent the reasoning process, transforming the optimization objective into maximizing the joint probability distribution $P(y, z \mid x)$. To construct a high-quality latent variable z, we define a dual-track reasoning paradigm. Specifically, we decompose the reasoning chain z into a sequence of ordered reasoning steps $\{s_1, s_2, \ldots, s_T\}$, wherein each step s_t is a composite structure incorporating both natural language intent and formal mathematical expression. This process can be formally represented as:

$$P(y \mid x) = \sum_{z \in \mathcal{Z}} P(y \mid x, z) \prod_{t=1}^{T} P(s_t \mid x, s_{<t}) \tag{1}$$

Here, $s_t = (I_t, F_t)$, where I_t denotes the natural language logical explanation at step t (e.g., "differentiate the function to determine monotonicity"), and F_t represents the corresponding LaTeX mathematical derivation (e.g., $f'(x) = \ldots$). Through this dual-encoding mechanism, we encourage the model to establish a robust alignment between mathematical symbols and their underlying semantic logic while learning symbolic operations. This approach may contribute to improved interpretability and robustness during reasoning processes.

In practical implementation, we utilize GPT-5 as the teacher model and employ prompt engineering to induce it to generate reasoning processes containing explicit `<think>` tags. Fig. 3 illustrates the data augmentation paradigm employed in our core dataset. Unlike conventional Chain-of-Thought (CoT) approaches, we prompt the teacher model to engage in self-reflection and trial-and-error within the `<think>` space. Specifically, before outputting a final conclusion, it is encouraged to explicitly demonstrate the exploration of implicit problem conditions and the evaluation of potential solution paths. This strategy introduces process supervision signals into the training data. To ensure data standardization, we further implemented format cleansing algorithms. Regular expressions were employed to uniformly convert all mathematical

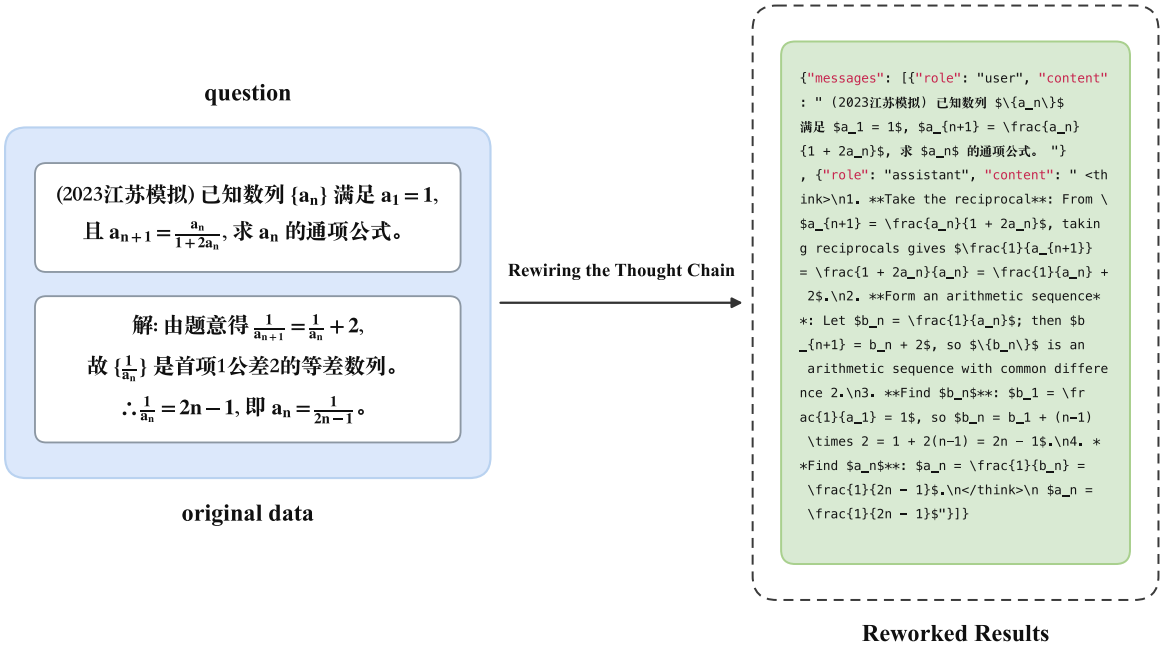


Fig. 3. Schematic diagram of the chain-of-thought reconstruction process. The left side shows the original problem—briefly analyzed data—while the right side displays the reconstructed high-density CoT sample. By explicitly introducing < think > tags, implicit problem-solving logic is transformed into structured reasoning steps, providing procedural supervision for the model.

symbols to standard LaTeX format, while filtering out samples where the reasoning process contradicted the final answer.

3.2. Synergistic tuning strategy

Constructing a high-quality difficulty-aware core-set is an important step toward achieving low-resource activation. Under low-resource constraints with only hundreds of samples, directly applying traditional parameter fine-tuning methods (such as Standard LoRA) (Hu et al., 2022) faces two major challenges: geometric constraints in the parameter update space may limit the model's search capability on complex mathematical manifolds, potentially leading to underfitting; and the high variance arising from minimal samples may cause the model to memorize specific problem types, potentially resulting in overfitting rather than generalized reasoning. To address these issues, we propose a synergistic fine-tuning strategy featuring internal decoupling and external perturbation. By incorporating Weight-Decomposed Low-Rank Adaptation (DoRA) (Liu et al., 2024) and Noise Embedding Fine-Tuning (NEFTune) (Jain et al., 2023), this approach aims to enhance the model's generalization robustness in both parameter and feature spaces.

3.2.1. Amplitude-direction decoupled parameter update mechanism

Although LoRA (Hu et al., 2022) reduces memory consumption through low-rank matrix decomposition, it assumes that weight updates occur entirely within the low-rank subspace. This results in strong coupling between the magnitude and direction of the weight matrix during updates. During full-fledged fine-tuning, models can independently adjust parameter direction (feature selection) and magnitude (feature strength). LoRA's coupling constraints limit this flexibility, revealing inductive bias limitations when handling tasks like mathematical reasoning that are highly sensitive to parameter geometry.

To address this issue, we employ the DoRA (Liu et al., 2024) algorithm to fine-tune the base model. The core concept of DoRA involves mathematically decoupling the pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, decomposing it into an amplitude vector $m \in \mathbb{R}^{1\times k}$ and a direction matrix $V \in \mathbb{R}^{d\times k}$. During fine-tuning, we freeze the pre-trained direction matrix V_0 and learn incremental direction changes ΔV using low-rank matrices B and A, while permitting full updates to the magnitude vector m. This update process can be formalized as follows:

$$W = m \odot \frac{V_0 + BA}{\|V_0 + BA\|_c} \tag{2}$$

Here, $\odot$ denotes broadcast multiplication, while $\|\cdot\|_c$ represents the vector norm computed along columns. Through this decomposition mechanism, the model can simulate a geometric update trajectory approaching that of full-scale fine-tuning while maintaining a low parameter count. For mathematical reasoning tasks, this decoupling property offers advantages: it allows the model to adjust the direction of feature extraction during early fine-tuning to adapt to new paradigms, while refining the magnitude of adjustments through task-specific optimization. This may help maximize feature extraction efficiency for the core dataset.

3.2.2. Manifold smoothing regularisation based on noise injection

Building upon parameter space optimization, we introduce the NEFTune (Jain et al., 2023) strategy into the feature space to further address the overfitting challenge in few-shot learning. While conventional regularization methods typically act upon the loss function or parameter weights, NEFTune directly influences the model's input representations. Its core principle involves injecting dynamically generated uniform distribution noise into the model's embedding layer, which serves as a form of implicit data augmentation.

Specifically, given the original sequence of embedding vectors $X_{\text{emb}} = \{e_1, e_2, \ldots, e_L\}$ for an input sequence, we introduce a noise vector ϵ of the same dimension as the original vectors during the forward propagation process. The perturbed vector X'_{emb} is computed as follows:

$$X'_{\text{emb}} = X_{\text{emb}} + \alpha \cdot \frac{\epsilon}{\sqrt{L \cdot d}}, \quad \epsilon \sim \text{Uniform}(-1, 1) \tag{3}$$

Here, α denotes the noise intensity hyperparameter, L represents the sequence length, and d signifies the hidden layer dimension. The scaling factor $\frac{1}{\sqrt{L\cdot d}}$ ensures that the noise energy does not increase linearly with sequence length, helping to preserve the stability of semantic information.

From the perspective of manifold learning, the injected random noise transforms each discrete training sample from a fixed point in feature space into a high-dimensional semantic neighborhood centered around its original features. This simulates the minute shifts in characteristics that occur when the same mathematical problem is expressed in different linguistic formulations or subjected to varying numerical perturbations. By encouraging the model to map these noisy features onto the same inference path z and final answer y, we artificially smooth the manifold distribution of training data, which may help extend the model's decision boundary. The robust fitting capability provided by DoRA (Liu et al., 2024) and the manifold smoothing mechanism offered by NEFTune form complementary effects during training. Together, they may help the model trained on the compact core set Math-Core-700 achieve improved accuracy while mitigating overfitting.

3.2.3. Synergistic mechanism of parameter decoupling and manifold smoothing

The CODER framework proposed in this study is not a simple superposition of DoRA (Liu et al., 2024) and NEFTune (Jain et al., 2023), but rather a synergistic combination built upon the complementary nature of their optimization dynamics. In scenarios involving mathematical fine-tuning with sparse data, the optimization process faces two challenges: insufficient expressive capability and the risk of convergence toward sharp minima. While relying solely on DoRA enhances the search freedom within the parameter space, the model may become susceptible to overfitting superficial features of the training data when samples are sparse. Conversely, using NEFTune alone can smooth decision boundaries through perturbation; however, without a flexible parameter update mechanism, the introduced noise may overwhelm the faint logical signals, potentially leading to underfitting. Therefore, a mechanism is needed that encourages the model to seek robust solutions while providing sufficient geometric freedom to capture high-precision inference pathways.

From an optimization geometry perspective, the random perturbation ϵ introduced by NEFTune at the embedding layer performs implicit curvature regularization on the loss function. Assuming the original loss function is $\mathcal{L}(x;\theta)$, the expected loss after noise introduction can be approximated by the second-order Taylor expansion at point x:

$$\mathbb{E}_{\epsilon}[\mathcal{L}(x+\epsilon;\theta)] \approx \mathcal{L}(x;\theta) + \frac{\alpha^2}{2}\,\mathrm{Tr}(\mathbf{H}_x) \tag{4}$$

Here, $\mathbf{H}_x$ denotes the Hessian matrix of the loss function with respect to the input, and α represents the noise intensity. The above formula indicates that NEFTune introduces a penalty term related to the trace of the Hessian matrix upon the original loss function. This regularization mechanism encourages the optimizer to avoid sharp valleys characterized by extreme curvature and high sensitivity to minor input variations, instead favoring flatter minima where the loss surface exhibits gentler gradients. For mathematical reasoning tasks, this is consistent with the model becoming less dependent on specific lexical patterns or numerical combinations within the problem statement, and potentially acquiring more robust patterns that are invariant to certain semantic perturbations, which may contribute to improved generalization.

However, seeking optimal solutions on smooth manifolds imposes stringent demands on parameter update mechanisms. Traditional LoRA (Hu et al., 2022), by strictly confining weight updates within coupled low-rank subspaces, may lack sufficient geometric degrees of freedom to accommodate the complex constraints imposed by Hessian trace penalties, potentially leading to optimization stagnation. DoRA's amplitude-direction decoupling mechanism plays a role here: it allows independent adjustment of the amplitude vector m to dynamically control the signal-to-noise ratio (SNR) of feature responses, which may help counteract potential signal attenuation from NEFTune noise. Simultaneously, the model employs low-rank matrices to perform precise rotations of the direction matrix V, helping to align with the smoothed manifold. In summary, NEFTune provides generalization-oriented constraints for the optimization process, while DoRA offers expressive pathways to navigate noise interference. This dual mechanism of feature space smoothing coupled with parameter space decoupling may help address the performance bottleneck in small-sample learning, enabling the model to achieve improved reasoning capabilities and robust generalization under the constraint of only 700 samples.

3.2.4. Training objectives and instruction

The model's optimization objective employs a standard autoregressive language modeling loss function. To focus on cultivating reasoning capabilities, we implement an instruction masking strategy when computing cross-entropy loss. Let the input sequence be $X = [x_{\text{ins}}, x_{\text{resp}}]$, where x_{ins} represents the user's prompt and question, and x_{resp} denotes the model-generated reasoning chain and answer. The loss function is computed solely on the x_{resp} component:

$$\mathcal{L}(\theta) = -\sum_{t=|x_{\text{ins}}|+1}^{|X|} \log P(x_t \mid x_{<t};\theta_{\text{DoRA}}) \tag{5}$$

This strategy ensures that gradient updates are focused on generating the reasoning logic and deriving the answer, rather than on restating the input question. This may help improve the signal-to-noise ratio and efficiency of fine-tuning.

4. Experiments

This chapter aims to validate the effectiveness of the CODER framework in activating mathematical reasoning capabilities within large language models under low-resource constraints through a multidimensional empirical evaluation. We designed a comprehensive assessment system encompassing general benchmarks, domain-specific benchmarks, and out-of-distribution benchmarks. By contrasting our approach with traditional fine-tuning paradigms reliant on extensive data, we aim to highlight the effectiveness of the high-quality core-set combined with collaborative regularization strategies.

4.1. Experimental setup and benchmark selection

Models. We employ two representative foundation models, Qwen2.5-Math-7B-Base and Llama-3.1-8B, to benchmark our approach. Qwen2.5-Math serves as an ideal candidate due to its specialized bilingual mathematical proficiency, while Llama-3.1 provides a critical baseline for assessing general-to-mathematical reasoning transfer.

Comparative Methods. To position our method against established practice, four reference groups were constructed. The first three reproduce each prior work's own standard data and configuration. The factor-controlled comparisons that isolate individual effects are reported separately in Section 4.4.

1. Base Model(Few-Shot): Serving as an un-fine-tuned baseline, this model employs a few-shot prompting strategy for reasoning. This benchmark assesses the foundational model's inherent knowledge reservoir and the lower bound of its reasoning capability without instruction alignment.
2. Base + MATH-LoRA: The base model undergoes LoRA fine-tuning using the industry-authoritative, competition-grade MATH dataset. This group represents conventional fine-tuning levels based on large-scale, high-difficulty English data, serving to contrast cross-lingual and complex logical reasoning capabilities.
3. Base + TAL-SCQ5K-LoRA: Performs LoRA fine-tuning using TAL-SCQ5K (an open-source Chinese mathematics and science dataset from TAL Education Group). This group represents the fine-tuning level based on large-scale Chinese domain data, serving as a crucial benchmark for measuring mathematical alignment effectiveness in Chinese contexts.

4. Ours (CODER): the method proposed in this study. It uses only 700 manually curated and cleaned high-quality examples spanning the three difficulty tiers, fine-tuned with DoRA and NEFTune (§3.2). This group tests the data-efficiency hypothesis.

Implementation Details and Hyperparameters. To ensure the reproducibility of our experiments, this study provides a full disclosure of the training configurations. All fine-tuning experiments were conducted on NVIDIA RTX GPUs using the `transformers` and `peft` libraries in `bfloat16` precision, with gradient checkpointing enabled to optimize memory efficiency. For both the LoRA baseline and the CODER (DoRA) framework, we targeted all linear projections—including q, k, v, o_proj and all MLP gate/up/down layers—with a consistent rank $r = 64$. Specifically, the standard LoRA baseline utilized an $\alpha_{lora} = 128$, whereas CODER adopted $\alpha_{lora} = 32$ to facilitate stable decoupled optimization of magnitude and direction within the DoRA mechanism. We employed the AdamW optimizer with a base learning rate of 2×10^{-4}, a cosine learning rate scheduler, and a warmup ratio of 0.1 over 5 training epochs. To mitigate potential overfitting on our compact 700-sample dataset, we integrated NEFTune with a noise intensity of $\alpha_{nef} = 5$ at the embedding layer and utilized an assistant-only loss objective to focus gradients exclusively on reasoning responses. The total effective batch size was maintained at 16 (batch size of 2 with 8 accumulation steps).All experiments on the non-competition benchmarks were repeated with three independent random seeds (42, 123, and 2024); for these we report the mean and standard deviation across the three seeds. The high-difficulty competition benchmarks (AMC 23, CNMO-CN, and Hard-CN) were evaluated on a single training seed: their evaluation already requires sixteen generations per problem per configuration, and repeating it across seeds would multiply an already substantial inference cost without changing any conclusion. For these we report Pass@1 / Pass@16 under a matched sampling budget, and because they rest on a single seed no across-seed deviation is defined. In all cases the checkpoint obtained after exactly 5 epochs was used, with no validation-based early stopping and no checkpoint selection.

Evaluation. This experiment established a multi-tiered assessment framework encompassing internationally recognized benchmarks, Chinese domain-specific benchmarks, and fine-grained capability maps to comprehensively examine the mathematical reasoning boundaries of the model. All test datasets underwent secondary verification to exclude questions identical or similar to those in Math-Core-700, helping to prevent data contamination and ensure testing fairness. For specific measures, please refer to Section 4.6.

1. **International General Reasoning Ability**. To verify whether the model has acquired more universal mathematical reasoning that is less dependent on language-specific patterns, we selected GSM8K (Cobbe et al., 2021) and AMC 23 as benchmark tests. GSM8K serves as a standard benchmark for examining foundational multi-step computational consistency, aiming to investigate whether models retain robust multi-step abilities after fine-tuning on higher-order complex data (such as final-stage university entrance examination questions), thereby quantitatively assessing potential catastrophic forgetting risks. AMC 23, encompassing competitions such as AMC 10/12 and AIME, represents more advanced general reasoning challenges that require extended multi-step deduction. These benchmarks test whether models exhibit transferable reasoning patterns when confronting problems with higher logical density than the training data.
2. **Domain Adaptation and Complexity Assessment**. Given the inclusion of Chinese Gaokao mock examination questions in the training data, we employ CMATH (Wei et al., 2023) and Gaokao-Math (selecting only fill-in-the-blank questions) (Zhong et al., 2024b) for testing. This section aims to assess the model's semantic comprehension within a Chinese linguistic context and its adaptability to Chinese standardized examination question formats, with a particular focus on performance in complex computations and specific knowledge domains (e.g., analytic geometry, derivatives).
3. **Fine-grained and Dynamic Generalization Capabilities**. To delineate detailed capability boundaries and eliminate data memorization interference, we employ C-Eval (Huang et al., 2023) and LiveMathBench (Liu et al., 2025) for in-depth validation. Through C-Eval's four major mathematical subsets (junior secondary, senior secondary, university mathematics, and probability statistics), we can assess the model's knowledge coverage at multiple levels. The results will help assess whether the proposed difficulty pyramid data construction strategy facilitates the activation of higher-order abstract concepts. Concurrently, LiveMathBench—incorporating the latest 2024 competition problems—serves as a decontamination test. As this dataset was released after the pre-training cutoff for the base model, it provides a basis for assessing whether the model has acquired transferable logical rules when confronting previously unseen heterogeneous challenges.

Evaluation Protocol and Prompting Conditions. Evaluations are run under two prompting conditions, and they differ in a direction that is unfavorable to the proposed method. The un-fine-tuned baselines are evaluated with five-shot prompting, that is, with demonstrations supplied alongside the instruction. Every fine-tuned model—Base + MATH-LoRA, Base + TAL-SCQ5K-LoRA and CODER—is evaluated zero-shot: the input consists of a single system instruction, "Please reason step by step, and put your final answer within `\boxed{}`.", followed by the problem statement, and no demonstrations or additional prompting stage are introduced.

4.2. Analysis of experimental results

This section details the performance metrics of the CODER framework. To establish representative baselines, the General SFT control group employs the MATH dataset (SFT1) and the TAL-SCQ5K dataset (SFT 2) as large-scale comparison bases. We contrast our method—trained on only 700 samples—against these large-scale data paradigms. This aims to evaluate our core hypothesis: whether, at one-tenth of the baseline data volume, a difficulty-aware core set combined with collaborative regularization remains competitive with each baseline's standard recipe; the controlled evidence for data selection and for the adapter–regularization interaction is given in Section 4.4. Evaluation will unfold across four dimensions: internationally standardized inference, domain adaptability, knowledge hierarchy, and temporal generalization, systematically assessing the effectiveness of our low-resource fine-tuning strategy. Furthermore, at the conclusion of this section, we conduct efficiency and resource constraint analyzes, highlighting the practical advantages of our approach in low-computational-power scenarios.To evaluate performance under a larger inference budget, we employ the Pass@16 metric alongside the standard Pass@1 metric for high-difficulty competition benchmarks (AMC 23, CNMO-CN, and Hard-CN). For Pass@16, we adopt a sampling temperature of $T = 0.7$ and $top_p = 0.95$, generating $N = 16$ independent reasoning paths. The final answer is extracted via regular expressions from the LaTeX `\boxed{}` tags. A problem is considered correct if at least one of the 16 generated answers matches the ground truth. In contrast, Pass@1 represents single-shot accuracy and is obtained using greedy decoding ($T = 0$ and $top_p = 1.0$). For all other standard datasets, we report the mean ± standard deviation to reflect statistical robustness.

4.2.1. Comparison of accuracy rates across major general-purpose test datasets

Performance on General Mathematical Reasoning

Across the two benchmarks, CODER improves both GSM8K and AMC 23, whereas the SFT baselines tend to trade GSM8K accuracy for higher AMC 23 Pass@16 (Fig. 4; Table 1). On GSM8K, Qwen's few-shot baseline scores 90.0%, while SFT1 and SFT2 fall to 75.0% and 56.6%,

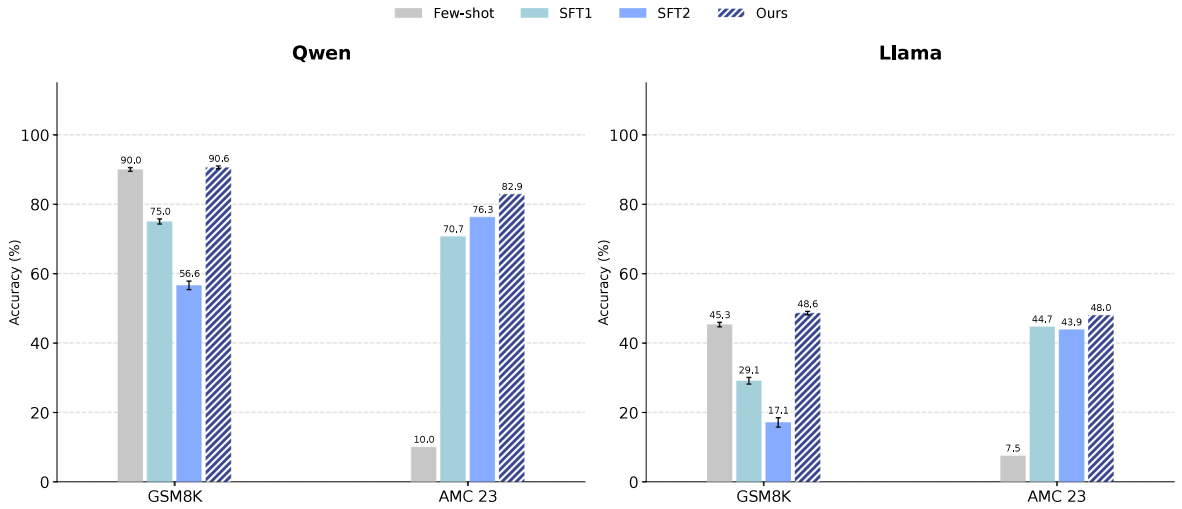


Fig. 4. Performance comparison of Qwen and Llama on the GSM8K (Accuracy) and AMC 23 (Pass@16) test sets. Our CODER framework consistently achieves superior reasoning accuracy across different base architectures. Note: for the high-difficulty AMC 23 dataset, Pass@16 is shown to illustrate the activated reasoning potential; for detailed Pass@1 vs. Pass@16 comparative metrics, please refer to Table 1.

Table 1
Detailed performance metrics for domain adaptation on AMC23 and GSM8K.GSM8K results are reported as the mean ± standard deviation over the three independent training seeds. AMC 23 results are reported as Pass@1 / Pass@16 from a single training seed under a matched sampling budget. On AMC 23 the few-shot baseline returns the same answer across all 16 samples for every problem, so Pass@1 and Pass@16 coincide.

Base Model	Method	GSM8K	AMC 23 (P@1 / P@16)
Qwen	Baseline (few-shot)	90.0±0.4	10.0 / 10.0
	General SFT1	75.0±0.6	15.0 / 70.7
	General SFT2	56.6±1.1	12.0 / 76.3
	Ours (CODER)	**90.6±0.3**	**40.0 / 82.9**
Llama	Baseline (few-shot)	45.5±0.5	7.5 / 7.5
	General SFT1	29.1±0.8	12.5 / 44.7
	General SFT2	17.1±1.2	10.8 / 43.9
	Ours (CODER)	**48.6±0.4**	**22.5 / 48.0**

Table 2
Detailed performance metrics for domain adaptation on CMATH and Gaokao-Math. The results are reported as the mean ± standard deviation of independent trials. The bold values indicate peak performance, and our CODER approach is highlighted with a light green background.

Base Model	Method	CMATH	GaoKao-Math
Qwen	Baseline (few-shot)	85.0 ± 0.5	57.6 ± 0.8
	General SFT1	83.0 ± 1.1	42.0 ± 1.3
	General SFT2	90.0 ± 0.6	60.0 ± 0.9
	Ours (CODER)	**93.6 ± 0.3**	**79.0 ± 0.4**
Llama	Baseline (few-shot)	0.0 ± 0.0	2.5 ± 0.4
	General SFT1	23.5 ± 1.2	6.0 ± 1.1
	General SFT2	21.0 ± 1.4	8.4 ± 1.2
	Ours (CODER)	**48.5 ± 0.7**	**20.0 ± 0.6**

respectively; for Llama, the baseline decreases from 45.5% to 29.1% and 17.1%. CODER reaches 90.6% on Qwen and 48.6% on Llama, exceeding the corresponding few-shot baselines.

On AMC 23, CODER achieves Pass@1/Pass@16 of 40.0/82.9 on Qwen and 22.5/48.0 on Llama, versus Pass@1 of 12.0–15.0% and 10.8–12.5% for the SFT baselines. As Fig. 5 shows, CODER stays ahead across the sampled k. Its margin over the best SFT variant is largest at $k = 1$ (+25.0 and +10.0 points for Qwen and Llama, respectively) and narrows at $k = 16$ (+6.6 and +3.3 points). These results suggest that the gain is more closely associated with single-sample reliability than with raising the sampling ceiling.

Domain Adaptation and Robustness Evaluation

In Chinese-language contexts and adaptive testing of high-difficulty integrated questions, CODER shows improved domain-transfer performance relative to conventional fine-tuning. As illustrated in Fig. 6, differences emerge on the Gaokao-Math and CMATH test sets.

For Qwen, General SFT1 decreases from a baseline of 57.6% to 42.0% on Gaokao-Math, while SFT2 reaches 60.0%, still below CODER's 79.0%. On CMATH, CODER improves from the baseline 85.0% to 93.6%, outperforming both SFT variants.

A similar pattern is observed for Llama. The few-shot baseline remains near zero (CMATH 0.0%, Gaokao-Math 2.5%), and SFT yields only modest gains (at best 23.5% on CMATH and 8.4% on Gaokao-Math). CODER achieves 48.5% on CMATH and 20.0% on Gaokao-Math.

These findings suggest that fine-tuning on large-scale English or generic mathematical corpora can, in some cases, lead to limited or even negative transfer. In contrast, a difficulty-aware core-set that incorporates Gaokao final-stage questions appears to facilitate more effective domain alignment under low-resource conditions.

Evaluation of Knowledge Hierarchy

To assess the difficulty hierarchy strategy, we analyze four mathematical subdomains in C-Eval (Table 3). For Qwen, CODER reaches 47.3% on Advanced Mathematics, above the strongest SFT variant (36.8%). It attains 72.0% on High School Mathematics and 66.7% on Probability & Statistics, exceeding the corresponding SFT results, while its Middle School Mathematics score (89.0%) is slightly below SFT1 (94.4%). For Llama, few-shot performance is near zero across subdomains. CODER obtains 16.6% on Advanced Mathematics, 27.7% on Middle School, 22.2% on High School, and 27.7% on Probability & Statistics. These scores exceed the SFT variants in Advanced Mathematics, Middle School, and Probability & Statistics, whereas SFT2 remains higher on High School (27.8% vs. 22.2%).

The improvements are compatible with the hypothesis that exposure to university-level abstract primitives may serve as a cognitive scaffold, supporting the transition from concrete computation to higher-order deduction. It is worth noting that CODER's Middle School

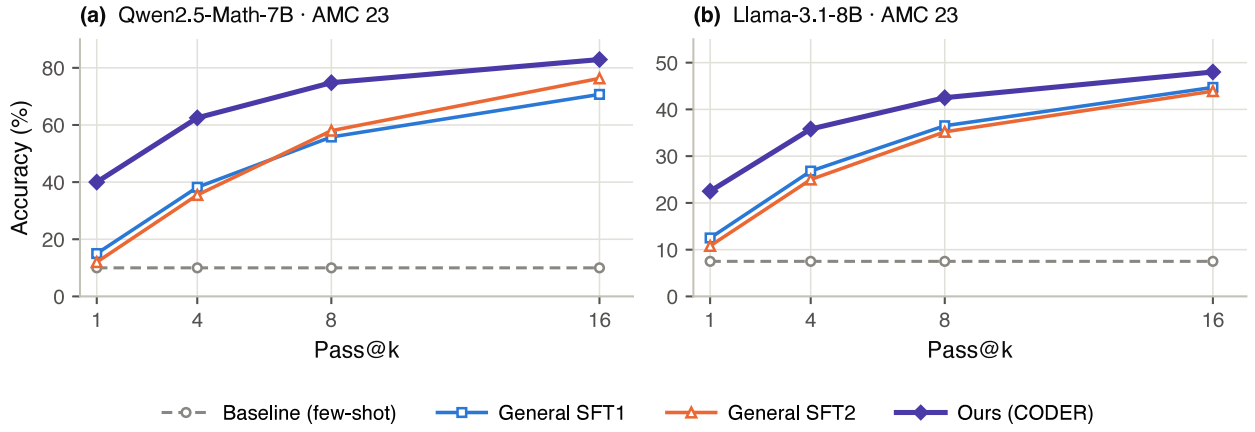


Fig. 5. Pass@k accuracy (k = 1, 4, 8, 16) on AMC 23 under a matched sampling budget. All points are derived from the same N = 16 generation set per problem; no additional inference was performed.Curves are obtained from a single training seed.

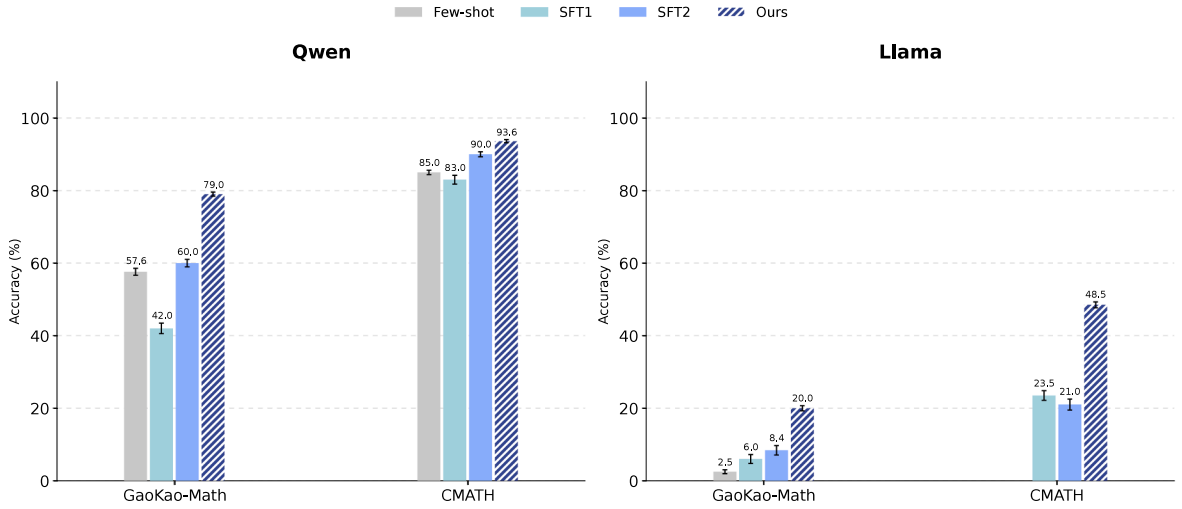


Fig. 6. Performance evaluation on high-difficulty Chinese mathematical reasoning tasks. The results demonstrate that our CODER framework significantly bridges the reasoning gap in complex, language-specific domains.

Table 3
Fine-grained performance comparison on C-Eval mathematical subdomains. Results are reported as the mean ± standard deviation over the three independent training seeds. The bold values indicate the highest performance within each model series, and our CODER framework is highlighted with a light green background.

Base Model	Method	Adv. Math	Middle School	High School	Prob. & Stat.
Qwen	Baseline (few-shot)	31.5 ± 0.8	84.2 ± 0.7	58.8 ± 0.9	50.0 ± 1.1
	General SFT1	36.8 ± 1.2	**94.4 ± 0.6**	44.4 ± 1.3	55.5 ± 1.2
	General SFT2	26.3 ± 1.4	83.3 ± 1.3	44.4 ± 1.5	47.0 ± 1.4
	Ours (CODER)	**47.3 ± 0.3**	89.0 ± 0.5	**72.0 ± 0.4**	**66.7 ± 0.5**
Llama	Baseline (few-shot)	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
	General SFT1	5.5 ± 1.5	10.5 ± 1.6	10.5 ± 1.4	16.7 ± 1.5
	General SFT2	10.5 ± 1.3	21.1 ± 1.5	**27.8 ± 1.4**	11.1 ± 1.6
	Ours (CODER)	**16.6 ± 0.7**	**27.7 ± 0.8**	22.2 ± 0.5	**27.7 ± 0.6**

Mathematics score on Qwen trails the strongest SFT variant (94.4%), and its High School Mathematics score on Llama trails SFT2 (27.8%). This pattern—a modest trade-off between specialized memorization and broader generalization—appears in specific subdomains across both model families. Overall, the results suggest that a compact, difficulty-stratified set of 700 high-density samples can support the formation of more transferable reasoning patterns within the model's parameter space (see Fig. 7).

Temporal Generalization and OOD Testing

To assess out-of-distribution (OOD) generalization, we evaluate on LiveMathBench, which was released after the training cut-off, making training-data contamination unlikely (Figs. 8 and 9). CODER obtains the highest score in each comparison of Table 4 and remains ahead across the sampled Pass@k points in Fig. 9.

For Qwen, CODER reaches 70.4% on CCEE-CN, above the few-shot baseline (50.6%) and the strongest SFT variant (53.2%). On the

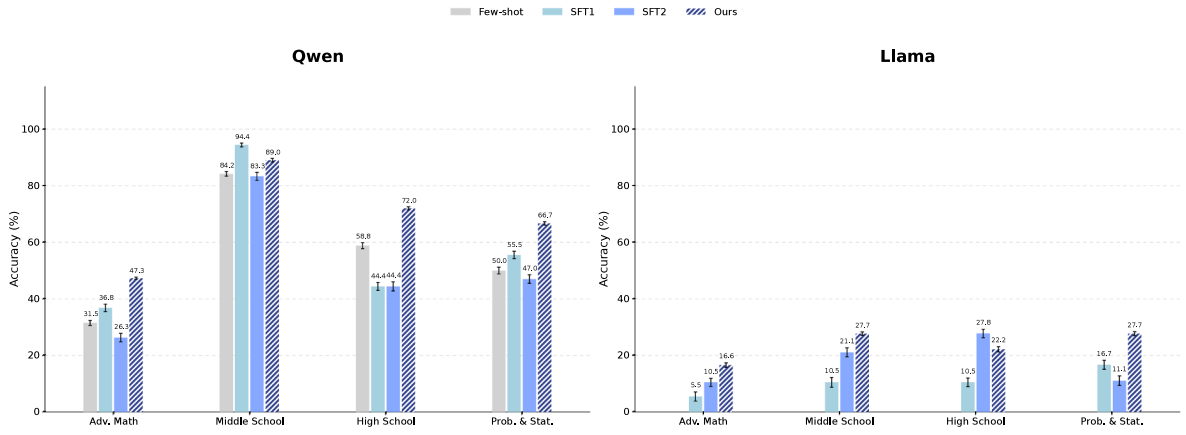


Fig. 7. Hierarchical capability assessment across four mathematical subdomains on C-Eval. Our framework achieves competitive or superior performance across all tiers, with notable gains in higher-order domains such as Advanced Mathematics and Probability & Statistics.

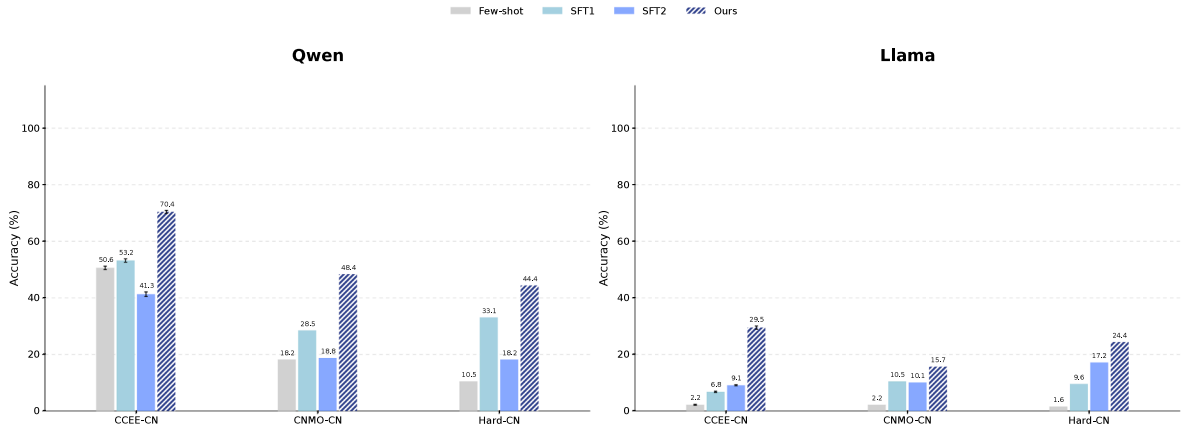


Fig. 8. Performance comparison on LiveMathBench across three difficulty tiers (CCEE-CN, CNMO-CN, Hard-CN). Our CODER framework consistently achieves superior reasoning accuracy across all complexity levels. Note: Pass@16 is reported for CNMO-CN and Hard-CN; see Table 4 for detailed Pass@1/Pass@16 metrics.

harder CNMO-CN and Hard-CN, it attains Pass@16 scores of 48.4 and 44.4, compared with 28.50 and 33.1 for the best SFT baselines. Llama follows a similar ordering: CCEE-CN improves from the best SFT result of 9.09% to 29.5%, and Pass@16 reaches 15.7 and 24.5 on CNMO-CN and Hard-CN, against 10.54 and 17.22 for the strongest SFT baselines.

The trend is not identical across suites. On AMC 23 the margin over the strongest baseline is largest at k = 1 and narrows as k grows, which is what one expects where the test set is small and a single problem represents a coarse fraction of the total; on the larger CNMO-CN and Hard-CN suites the margin instead widens with k, which is the opposite of what a decoding artifact would produce.Fig. 9 indicates that the advantage is not confined to single-sample decoding: the margin over the best SFT variant is larger at $k = 16$ than at $k = 1$ in each panel, increasing from +4.5 to +11.3 points on Qwen Hard-CN and from +16.6 to +19.9 on Qwen CNMO-CN, and from +3.9 to +5.2 on Llama CNMO-CN and +3.7 to +7.2 on Llama Hard-CN. Additional sampling therefore does not appear to close the gap, which would be expected if the gain were primarily a decoding artifact. The ordering holds on both backbones, suggesting that CODER's benefit is not tied to the training distribution and can transfer to unseen competition-level problems.

4.2.2. *Efficiency and resource constraints analysis*

All training reported in this work was carried out on a single consumer-grade device with a 24GB memory budget, using bfloat16 precision and gradient checkpointing throughout. Math-Core-700 is deliberately compact: it contains 700 instances, of which 10.0% exceed 1024 tokens under the Qwen2.5 tokenizer and 9.5% under the Llama-3.1 tokenizer, with a maximum of 4068 tokens (Appendix C). Peak memory is governed by these longest instances rather than by the corpus average; with a maximum sequence length of 4096 tokens, training the full curriculum with CODER allocated a peak of approximately 23GB, leaving the remainder of the budget to gradient accumulation rather than requiring a reduction in batch size or context length.

The efficiency of the framework is best measured in update steps rather than in per-step cost. Math-Core-700 contains roughly one tenth of the examples used by the MATH-LoRA baseline (7500) and one seventh of those used by TAL-SCQ5K-LoRA (5000); at a common effective batch size and the same number of epochs, the proposed method therefore performs approximately 9% of the optimisation steps required by the former. It should be noted that wall-clock time does not scale down proportionally, because core-set instances carry substantially longer reasoning chains than the average MATH instance (Appendix C): the saving is in the number of parameter updates, not in the cost of each one. The additional parameter overhead of DoRA's magnitude vector and the embedding-level noise of NEFTune are negligible by comparison, since neither alters the shape of the computation graph.

The practical consequence is that the full curriculum can be trained to convergence on a single consumer-grade card without lowering the batch size or truncating the context, which is what makes the data-efficiency claim of this work operationally meaningful rather than merely statistical.

Table 4
Detailed performance metrics on LiveMathBench. The results demonstrate the superior out-of-distribution reasoning capabilities of the CODER framework. CCEE-CN Results are reported as Mean ± Standard Deviation (%). For CNMO-CN and Hard-CN, results are reported as Pass@1 / Pass@16 to provide a comprehensive evaluation of both zero-shot reliability and maximum reasoning potential. The highest accuracy in each category is bolded. The CNMO-CN and Hard-CN columns are obtained from a single training seed, evaluated with Pass@1 / Pass@16 under a matched sampling budget of 16 generations per problem.

Base Model	Method	CCEE-CN	CNMO-CN (P@1 / P@16)	Hard-CN (P@1 / P@16)
Qwen	Baseline (few-shot)	50.6±0.5	11.10 / 18.25	4.00 / 10.53
	General SFT1	53.2±0.5	5.00 / 28.50	8.00 / 33.1
	General SFT2	41.5±0.7	11.10 / 18.83	12.50 / 18.25
	Ours (CODER)	**70.4±0.4**	**27.70 / 48.4**	**17.00 / 44.4**
Llama	Baseline (few-shot)	2.2±0.1	2.20 / 2.20	1.60 / 1.60
	General SFT1	6.82±0.1	4.55 / 10.54	4.10 / 9.61
	General SFT2	9.09±0.1	4.20 / 10.12	7.80 / 17.22
	Ours (CODER)	**29.5±0.5**	**8.40 / 15.7**	**11.50 / 24.5**

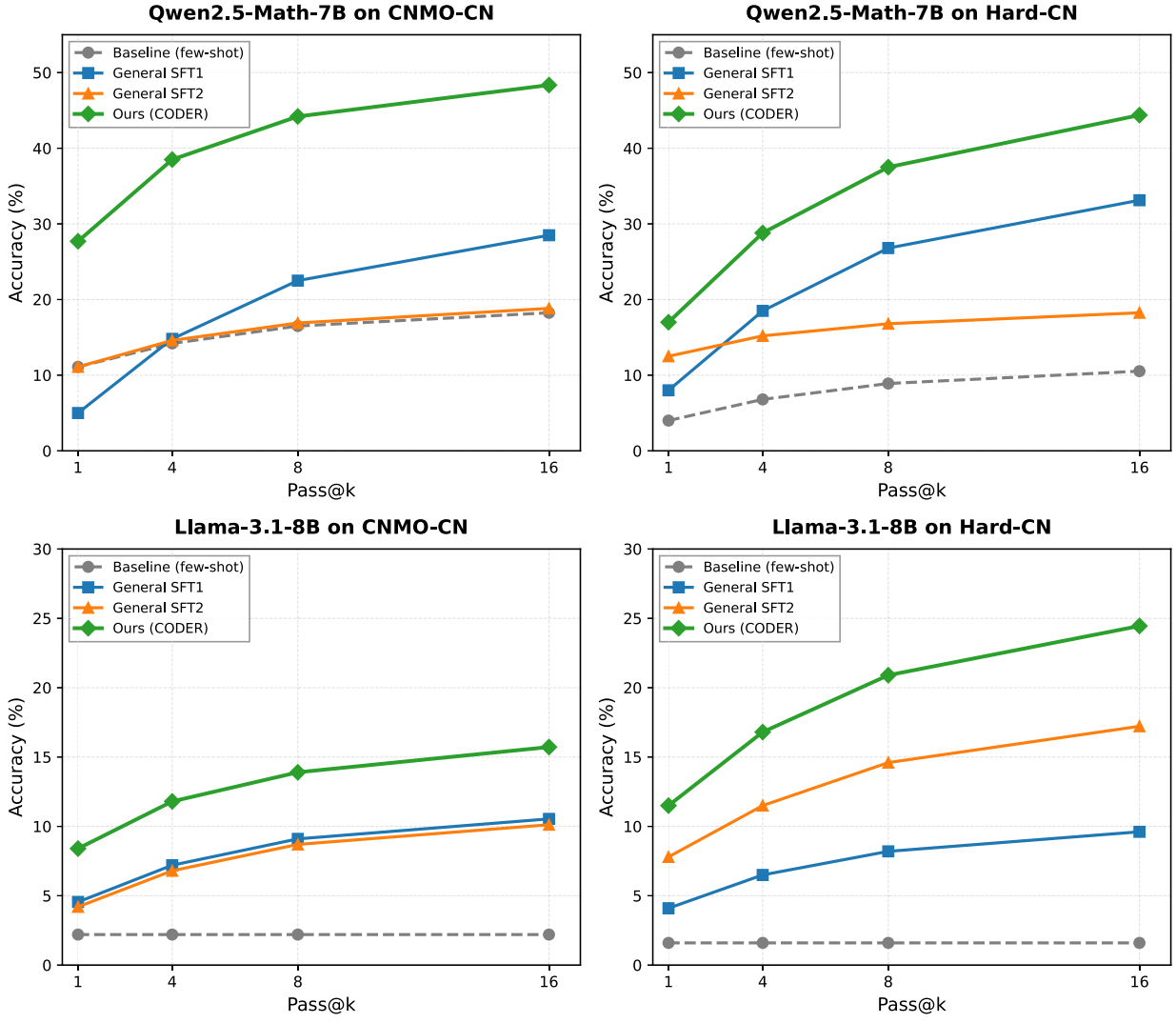


Fig. 9. Pass@k accuracy ($k = 1, 4, 8, 16$) on CNMO-CN and Hard-CN under a matched sampling budget. All points are derived from the same $N = 16$ generation set per problem; no additional inference was performed. CODER attains the highest accuracy at every k in all four panels, and its margin over the strongest SFT variant grows with additional sampling: the margin at $k = 16$ exceeds that at $k = 1$ in every panel, so the benefit is not confined to single-sample decoding.Curves are obtained from a single training seed.

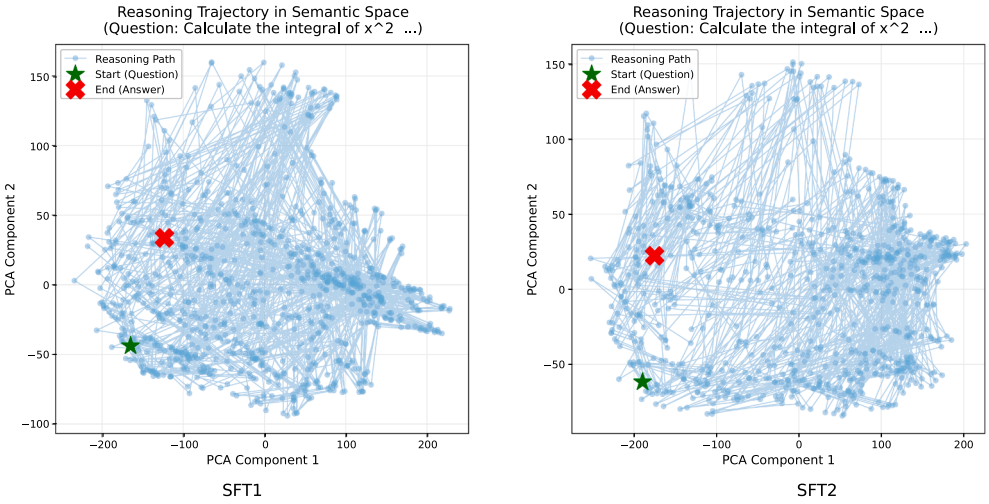


Fig. 10. Example diagram of inference trajectories for SFT 1 and SFT 2.

4.3. Mechanistic analysis of representation dynamics

Although the quantitative evaluation presented earlier confirms the CODER framework's outstanding performance across multiple benchmarks, accuracy metrics alone cannot fully reveal the underlying mechanisms of qualitative changes in model parameters. To validate whether the proposed fine-tuning strategy genuinely activates latent reasoning capabilities within the base model—rather than merely achieving surface-level fitting to training data—this section adopts a microscopic perspective within feature space. Through visualization techniques, it delves into the structure and dynamic evolution of the model's semantic representations.

4.3.1. Dynamics of reasoning trajectories

To further elucidate the divergent cognitive patterns exhibited by models during the generation of chains-of-thought, we extracted the hidden layer state sequences from different models solving the same calculus problem. These sequences were then visualized as reasoning trajectories via PCA (as depicted in Fig. 10). This comparative series of diagrams provides supportive observational evidence (via PCA trajectories as a proxy) that is consistent with our assertion that data quality can play a more critical role than quantity in this research setting.

As illustrated in Fig. 10 (left), the SFT-MATH model fine-tuned on the full MATH dataset exhibits a highly entropic, cluster-like distribution in its reasoning trajectories. The starting and ending points are surrounded by numerous disordered intermediate states, lacking a clear central axis. This trajectory, reminiscent of Brownian motion, suggests that despite exposure to extensive data, the model appeared to struggle in acquiring effective reasoning paradigms, potentially due to the absence of high-quality reasoning chains in the original data. During generation, it tends to engage in extensive exploration within the semantic space, reflecting a degree of uncertainty and discontinuity in its reasoning process. Fig. 10 (right) displays the trajectory of the SFT-TAL model fine-tuned on TAL data. Compared to SFT-MATH, this trajectory exhibits some directionality but overall presents a pronounced zigzag oscillation. Such lateral jumps suggest that the model undergoes noticeable semantic shifts at each inference step, struggling to establish a smooth transitional bridge between the question domain and the answer domain. This observation is consistent with the interpretation that traditional LoRA, when processing high-density Chinese data, may face limitations in feature extraction precision under certain conditions, potentially related to the coupled nature of its parameter updates, which may result in an inference process that exhibits frequent hesitation and corrective steps.

Fig. 11 illustrates the trajectory of the CODER model in this study, exhibiting a relatively structured spiral pattern:

1. Global Directionality: Trajectories exhibit a clear flow from origin to destination, reflecting the model's strong goal-directed tendency.
2. Local Structure: Within the macro-level linear trajectory, compact local spirals are embedded. These appear to represent structured reasoning rather than chaotic oscillations—each spiral potentially corresponding to specific steps in problem decomposition, formula derivation, or logical validation.
3. Manifold Smoothing: The trajectory lines exhibit fluid continuity, which is consistent with the regularization effect of NEFTune. This observation aligns with the interpretation that high-quality core data sets may contribute to forming a smoother reasoning trajectory within the model's semantic space, potentially facilitating more coherent multi-step logical transitions.

4.4. Controlled ablations: data selection and collaborative optimisation

Comparative experiment

To isolate the independent contributions of the difficulty-aware coreset and the collaborative fine-tuning strategy, we conduct a factorial-style ablation under strictly matched conditions. All experiments use identical 700-example budgets, rank, target modules, learning-rate schedule, and number of training epochs. For each combination of adapter (LoRA / DoRA) and regularization (with / without NEFTune). The scaling factor is deliberately not shared across the two parts of our study: the LoRA baselines in Tables 1–4 follow the standard configuration used in prior work ($\alpha_{lora} = 128$), whereas every variant in the ablation of Section 4.4 uses a single shared value ($\alpha_{lora} = 32$) so that no configuration differs from another in any hyperparameter.Sharing a single value removes any per-arm tuning advantage, but we do not claim it is the jointly optimal setting for either adapter. Both the difficulty-aware Math-Core-700 pyramid and a randomly sampled 700-example subset from MATH are trained with three independent random seeds (42, 123, 2024).

This design yields a factorial comparison (data selection × adapter × regularization) under controlled data volume, enabling us to examine main effects and their interactions. Results on C-Eval and LiveMathBench are reported in Tables 5 and 6.

The Random arms isolate the contribution of data distribution: `Random + Dora+nef` shares CODER's DoRA and NEFTune configuration and differs only in the training data. Across the eighteen comparisons on C-Eval and LiveMathBench, CODER attains higher scores in sixteen;

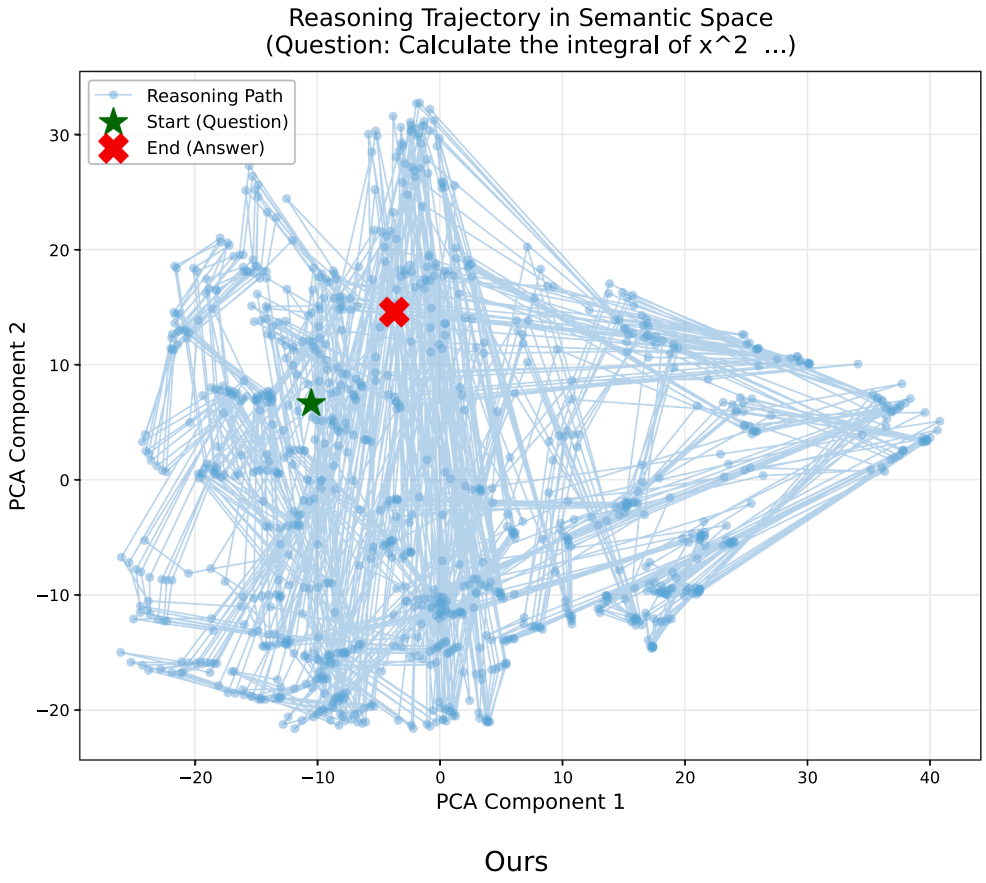


Fig. 11. Schematic diagram of the CODER method inference trajectory.

the two exceptions occur on C-Eval Middle School, the least demanding subset, where random sampling from MATH already covers this level relatively densely, so reallocating the fixed 700-example budget toward harder problems does not yield further improvement on this subset (Qwen: 94.4 vs. 89.0; Llama: 28.9 vs. 27.7).

The remaining gains are concentrated on the harder subsets. For Qwen, C-Eval Advanced Mathematics improves from 36.8 to 47.3 and High School Mathematics from 70.0 to 72.0; for Llama, Advanced Mathematics improves from 15.4 to 16.6 and Probability & Statistics from 25.2 to 27.7. On LiveMathBench, Qwen improves from 54.5 to 70.4 on CCEE-CN and from 40.8 to 48.4 on CNMO-CN Pass@16, while Llama improves from 19.5 to 29.5 on CCEE-CN, with corresponding gains on CNMO-CN and Hard-CN. These results suggest that difficulty-aware filtering redirects a fixed budget toward long-tail, university- and competition-level problems that random sampling tends to underrepresent.

A Study on the Ablation of Collaborative Optimisation Strategies

The four variants in Table 6 form a 2×2 factorial design over the adapter (LoRA vs. DoRA) and NEFTune, trained on the same Math-Core-700 data with identical hyper-parameters. CODER achieves the highest score across the nine reported metric columns for both backbones, so the observed gaps primarily reflect the adapter and regularization choices rather than the training data. On Qwen, the largest margins appear on C-Eval High School (72.0 vs. 64.7) and CNMO-CN Pass@16 (48.4 vs. 35.6); on Llama, they appear on C-Eval Middle School (27.7 vs. 22.2) and Probability & Statistics (27.7 vs. 21.5).

Neither component alone appears to account for the final gain. C-Eval High School is representative and behaves similarly across backbones: adding NEFTune to LoRA leaves the score nearly unchanged (Qwen: 50.0 vs. 50.0; Llama: 15.6 vs. 15.5), switching to DoRA alone raises it to 64.7 and 18.4, and the combination reaches 72.0 and 22.2. The two factors thus appear complementary rather than independently sufficient. In this low-sample regime, the magnitude-direction coupling of standard LoRA may restrict the directions along which the low-rank update can move, potentially limiting the extent to which the model absorbs dense reasoning chains; decoupling magnitude from direction relaxes this restriction, and NEFTune can then operate on a parameterisation able to exploit it.

4.5. An investigation into granularity error analysis and reasoning mechanisms

To examine whether the observed gains reflect enhanced logical reasoning rather than the mere restoration of arithmetic capability, this study conducted a fine-grained manual audit, drawing on the misclassification analysis of Zhong et al. (2024a). We randomly sampled 100 instances from the set of cases where the baseline models (Qwen-2.5-Math-Base, MATH-LoRA and TAL-LoRA) failed but CODER succeeded, and categorized them into three types: semantic misunderstanding (SM), logical gap (LG) and computational error (CE).

4.5.1. Quantitative analysis: distribution of corrections

As shown in Fig. 12, the improvement in CODER's performance is primarily driven by the resolution of higher-order reasoning failures, rather than simple computational corrections. Across all three benchmark comparisons, pure computational errors (CE) accounted for only a small minority (10.0%–19.0%) of the cases resolved. Conversely, over 80% of the improvements stem from the explicit mitigation of semantic misunderstandings (such as overlooking implicit constraints in the task) and logical gaps (such as incoherent derivation processes).

In particular, when compared with TAL-SCQ5K-LoRA, CODER corrected 57.1% of logical gaps. This observation is consistent with the

Table 5
Ablation on data selection strategy. All Random variants are trained on a quality-gated uniform sample of 700 examples drawn from the MATH training split, under identical hyper-parameter settings; "−nef" and "+nef" denote training without and with NEFTune, respectively. **Ours (CODER)** uses the difficulty-aware Math-Core-700 pyramid with DoRA + NEFTune, whereas "Random + Dora+nef" applies the same DoRA + NEFTune configuration to a random 700-example subset. The two therefore differ only in the training data, which isolates the effect of data selection. For CNMO-CN and Hard-CN, results are reported as P@1 / P@16.

Base Model	Method	Adv. Math	Middle School	High School	Prob. & Stat.
Llama	Random + Lora−nef	13.8±0.7	24.1±0.6	17.9±0.7	21.8±0.7
	Random + Lora+nef	14.6±0.9	25.3±0.8	18.6±0.8	23.1±0.8
	Random + Dora+nef	15.4±0.6	28.9±0.5	20.1±0.6	25.2±0.6
	Ours (CODER)	**16.6±0.7**	**27.7±0.8**	**22.2±0.5**	**27.7±0.6**
Qwen	Random + Lora−nef	33.2±0.6	88.7±0.5	62.4±0.6	55.8±0.7
	Random + Lora+nef	34.9±0.7	91.2±0.5	65.8±0.6	58.3±0.7
	Random + Dora+nef	36.8±0.4	94.4±0.3	70.0±0.4	61.1±0.5
	Ours (CODER)	**47.3±0.3**	**89.0±0.5**	**72.0±0.4**	**66.7±0.5**

Base Model	Method	CCEE-CN	CNMO-CN (P@1 / P@16)	Hard-CN (P@1 / P@16)
Llama	Random + Lora−nef	16.8±1.1	5.20 / 11.3	8.10 / 19.4
	Random + Lora+nef	18.1±1.0	5.60 / 11.9	8.70 / 20.3
	Random + Dora+nef	19.5±1.1	6.00 / 12.5	9.50 / 21.2
	Ours (CODER)	**29.5±0.5**	**8.40 / 15.7**	**11.50 / 24.5**
Qwen	Random + Lora−nef	48.7±0.9	19.40 / 36.2	13.20 / 34.8
	Random + Lora+nef	51.6±0.8	21.10 / 38.5	14.10 / 36.7
	Random + Dora+nef	54.5±0.8	22.80 / 40.8	15.00 / 38.5
	Ours (CODER)	**70.4±0.4**	**27.70 / 48.4**	**17.00 / 44.4**

Table 6
Ablation study of key components on Llama and Qwen under the same Math-Core-700 data and fully identical hyperparameters. The four variants therefore form a complete 2×2 factorial design over the adapter (LoRA vs. DoRA) and NEFTune.

Base Model	Method	Adv. Math	Middle School	High School	Prob. & Stat.
Llama	Lora−nef	12.5±0.7	20.2±0.5	15.6±0.7	17.3±0.6
	Lora+nef	15.8±1.1	22.2±0.9	15.5±0.6	20.8±0.8
	Dora−nef	15.3±0.8	22.2±0.7	18.4±0.9	21.5±0.6
	Ours (CODER)	**16.6±0.7**	**27.7±0.8**	**22.2±0.5**	**27.7±0.6**
Qwen	Lora−nef	42.1±0.4	73.6±0.3	50.0±0.4	44.4±0.5
	Lora+nef	36.8±1.2	84.2±0.4	50.0±0.6	61.1±0.9
	Dora−nef	36.8±0.9	84.2±0.5	64.7±0.7	61.1±0.8
	Ours (CODER)	**47.3±0.3**	**89.0±0.5**	**72.0±0.4**	**66.7±0.5**

Base Model	Method	CCEE-CN	CNMO-CN (P@1 / P@16)	Hard-CN (P@1 / P@16)
Llama	Lora−nef	18.7±1.2	4.80 / 12.5	8.30 / 20.7
	Lora+nef	22.2±1.2	4.20 / 10.9	4.80 / 11.1
	Dora−nef	25.4±1.2	4.80 / 11.1	7.20 / 17.7
	Ours (CODER)	**29.5±0.5**	**8.40 / 15.7**	**11.50 / 24.5**
Qwen	Lora−nef	54.5±0.8	18.50 / 35.6	16.50 / 38.65
	Lora+nef	61.4±0.8	11.10 / 22.2	4.80 / 12.5
	Dora−nef	68.1±0.6	8.00 / 16.6	4.80 / 12.5
	Ours (CODER)	**70.4±0.4**	**27.70 / 48.4**	**17.00 / 44.4**

interpretation that, compared to large-scale data aggregation, a difficulty-aware core-set filtering strategy may more effectively facilitate the establishment of robust reasoning paths, and may thereby help avoid logical breaks during complex derivation processes.

4.5.2. Qualitative analysis: the semantic buffering effect in dual-track reasoning

The Semantic Buffer Mechanism. To provide a qualitative understanding of how CODER mitigates semantic errors, we present a representative case involving vector algebra in Table 7. In this example, both MATH-LoRA and TAL-LoRA failed to identify or correctly process the algebraic expansion constraints, leading to a Logical Gap (missing terms) or Semantic Hallucination (confusing the problem with other exam questions). In contrast, CODER utilizes its dual-track reasoning—specifically the <think> track—as a Semantic Buffer. It explicitly parses the problem's conditions ($\vec{a} \cdot \vec{b} = b^2$ and $|\vec{a} - \vec{b}| = |\vec{b}|$) and performs a rigorous step-by-step logical derivation, ensuring that all constraints are satisfied before reaching the final execution phase.

Internalized versus Explicit Semantic Parsing.

The case study in Table 7 contrasts the baselines' failure modes under standard prompting—a Logical Gap (a dropped term in the algebraic expansion) and Semantic Hallucination (drift to an unrelated problem)—with CODER's coherent derivation, in which the problem's constraints are parsed by the model itself rather than supplied by an external prompt. Recent work by Zhong et al. (2024a) improves GSM8K through inference-time prompting alone, reporting 97.1% with GPT-4 (0613) under zero-shot prompting, while reporting substantially smaller gains on open backbones of the scale studied here. Its diagnosis aligns with ours:

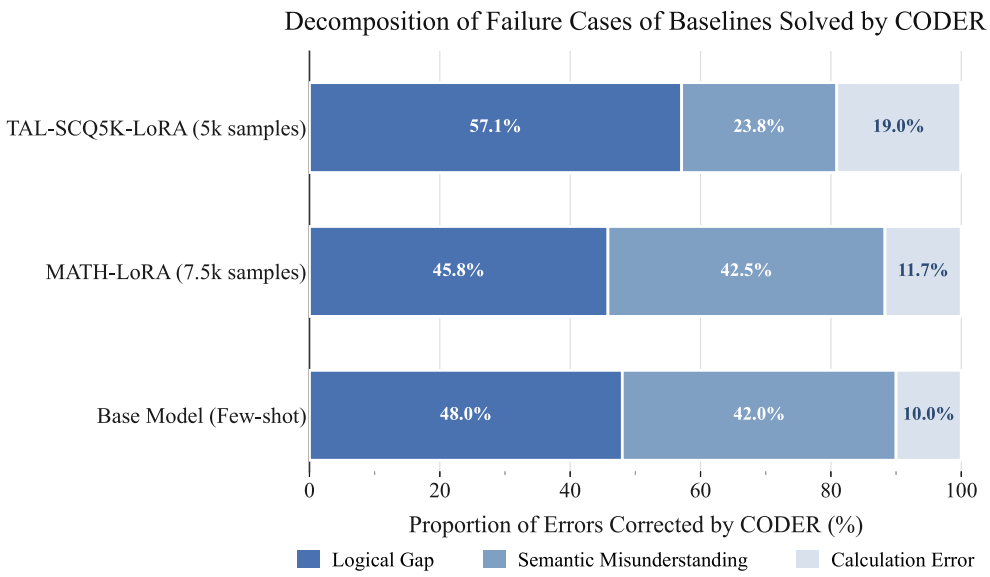


Fig. 12. Decomposition of error types in baselines corrected by CODER.

Table 7
Qualitative comparison of reasoning chains. Red bold text highlights logical gaps and hallucinations in baselines, while green bold text indicates CODER's rigorous logical steps.

Model	Reasoning Chain (Case: Vector Algebra Problem)
Question	Given vectors $\vec{a}, \vec{b}$ satisfy $\vec{a} \cdot \vec{b} = b^2$ and $\lvert\vec{a} - \vec{b}\rvert = \lvert\vec{b}\rvert$, what is the angle between $\vec{a}$ and $\vec{b}$?
Ours (CODER)	**[Success]** <*think*> 1. Utilize the magnitude relationship: $\mathbf{\lvert\vec{a} - \vec{b}\rvert^2 = \lvert\vec{b}\rvert^2 \Rightarrow \lvert\vec{a}\rvert^2 + \lvert\vec{b}\rvert^2 - 2(\vec{a} \cdot \vec{b}) = \lvert\vec{b}\rvert^2}$. 2. Substitute the given condition $\vec{a} \cdot \vec{b} = b^2$: $\mathbf{a^2 + b^2 - 2b^2 = b^2 \Rightarrow a^2 = 2b^2}$. 3. Calculate the cosine: $\cos\theta = \frac{\vec{a}\cdot\vec{b}}{\lvert\vec{a}\rvert\lvert\vec{b}\rvert} = \frac{b^2}{\sqrt{2}b \cdot b} = \frac{1}{\sqrt{2}}$. </*think*> **Result:** $\pi/4$
MATH-LoRA	**[Failure - Logical Gap]** Expanding $(\mathbf{a} - \mathbf{b})^2 = \mathbf{b}^2$ yields $\mathbf{a^2 - 2a \cdot b = 0}$. **(Critical Error: Missing $\mathbf{b}^2$ term)** Thus $\cos\theta = 1/2$, leading to 60°.
TAL-LoRA	**[Failure - Semantic Hallucination]** Assume $\mathbf{(\vec{a} - \vec{b})^2 = 2}$, then $\vec{a}^2 = 4$. **(Hallucinated Values)** ...(The model drifts into an unrelated 2019 exam problem about perpendicular vectors).

semantic misunderstanding is the dominant failure mode, and it is likewise the failure mode behind more than 80% of the errors CODER corrects. The two approaches therefore target the same bottleneck through different mechanisms, and are best regarded as orthogonal rather than competing: the prompt-based route renders the parsing explicit at inference time and leaves the weights untouched, whereas CODER internalizes the same parsing in the parameters during fine-tuning, so that the constraints are recovered without any prompt-level scaffolding. This is the distinction our results speak to directly: Section 4.8 shows that CODER already attains partial robustness to contextual distractors under standard prompting, that is, under a condition in which the parsing is not externally supplied.

4.6. Decontamination audit and leakage risk assessment

To mitigate the risk of data memorisation and ensure temporal independence, we have implemented a rigorous multi-stage decontamination pipeline. Crucially, this pipeline evaluates the complete Math-Core-700 dataset—including both the original problem statements and the newly reconstructed reasoning chains generated by GPT-5—against all evaluation benchmarks (e.g., MATH, AMC 23, CNMO-CN, Hard-CN, GSM8K, and Gaokao-Math).

First, we use MinHash LSH to perform a coarse-grained scan between Math-Core-700 and all evaluation benchmarks. Subsequently, we implemented a rigorous character-level decontamination review, concentrating on the two benchmarks that carry the largest and smallest performance deltas relative to the few-shot baseline (+21.4 points on Gaokao-Math and +0.6 points on GSM8K), where contamination, if present, would be most and least visible respectively. We employed the character-level Jaccard similarity metric, which is more robust for mathematical content than semantic embeddings, as the latter are often artificially inflated by repetitive LaTeX syntax. Based on the results, we compared samples with a similarity score exceeding 0.5 on a case-by-case basis; if a problem or its reasoning chain was found to be the original question or merely a numerical variation, we manually removed or reconstructed it.

Finally, we present Gaokao-Math as a representative case study to illustrate this rigorous process. We calculated the maximum similarity score for each test sample in Gaokao-Math relative to the entire Math-Core-700 training set. As shown in Fig. 13, the similarity distribution strictly follows a low-mean pattern ($\mu = 0.423$), and there are no instances that are completely or nearly completely overlapping. **Similar conclusions were rigorously verified across all other aforementioned benchmarks.**

- **Safety Threshold:** We set a safety threshold of 0.5. Approximately 92% of the test samples fall below this threshold; the few that exceed it were cleared by manual inspection.
- **Leakage Exclusion:** The maximum similarity found was 0.683, which, upon manual inspection, represents short problems sharing common mathematical notations but with entirely different logical targets. No samples exceeded the 0.7 High-Risk threshold.

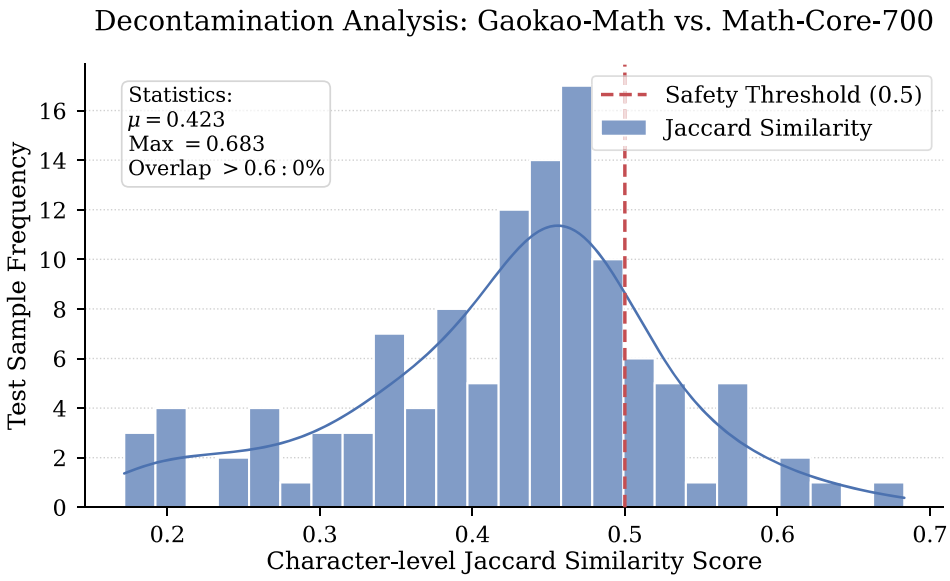


Fig. 13. Quantified Jaccard similarity distribution plot.

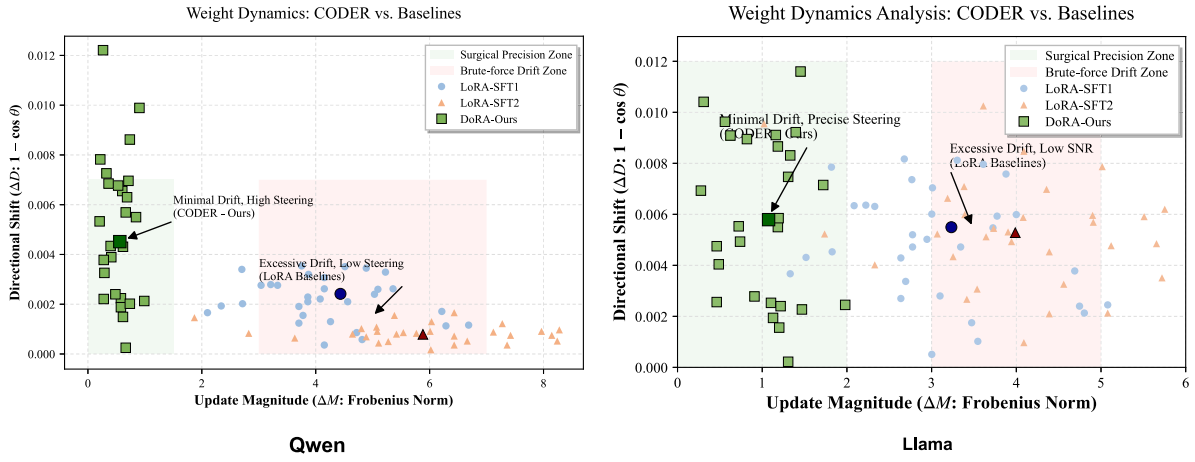


Fig. 14. Mechanistic analysis of weight dynamics: CODER vs. LoRA baselines the left panel shows results for Qwen, and the right panel shows Llama.

4.7. Mechanistic analysis of weight update geometry

To provide further analysis of the geometric patterns underlying CODER's competitive data efficiency and its cross-architectural consistency, we examine the fine-tuning process of both the math-specialized Qwen2.5-Math-7B and the general-purpose Llama-3.1-8B through the lens of Geometric Weight Dynamics. We utilize two complementary metrics: Update Magnitude (ΔM, Frobenius Norm) and Directional Shift (ΔD, $1 - \cos\theta$).

Rationale for Metric Selection. Grounded in the geometric decomposition of weight updates (Liu et al., 2024), ΔM quantifies the *parametric drift* from the pre-trained manifold, where excessive drift serves as a proxy for catastrophic forgetting. Conversely, ΔD characterizes the *semantic re-alignment* of the internal logic. An efficient framework should prioritize a high $\Delta D/\Delta M$ ratio—achieving substantial logical steering with minimal structural perturbation.

Cross-Model Quantitative Observations. As illustrated in Fig. 14, a notable contrast exists between CODER and standard LoRA baselines across both architectures.

- **In the math-specialized Qwen (Fig. 14 Left):** CODER exhibits a relatively low magnitude drift ($\Delta M \approx 1.07$) while achieving a directional shift comparable to or higher than LoRA baselines, which require approximately 3× more parametric drift ($\Delta M > 3.2$).
- **In the general-purpose Llama (Fig. 14 Right):** A similar pattern is observed. CODER maintains a low magnitude drift ($\Delta M \approx 0.56$) while producing a larger directional shift ($\Delta D \approx 0.0045$) than LoRA-SFT2.

This phenomenon is consistent with the interpretation that CODER may facilitate more targeted logical alignment by steering the weight vectors toward reasoning-relevant directions while largely preserving the integrity of the pre-trained manifold, regardless of whether the base model is already math-aligned.

Conclusion on Efficiency. In contrast, all LoRA baselines exhibit substantially larger magnitude drift (ΔM ranging from 3.2 to 5.8) while achieving comparatively limited directional alignment. This pattern suggests that LoRA baselines operate in a mode that heavily modifies parameters to fit 5k–7k samples, which may come at the cost of general reasoning foundations. The consistent behavior of CODER across different model families is consistent with the interpretation that its effectiveness reflects the synergy between its difficulty-aware core-set and decoupled optimization. This synergy may promote updates that align more closely with transferable reasoning rules, rather than surface-level pattern matching.

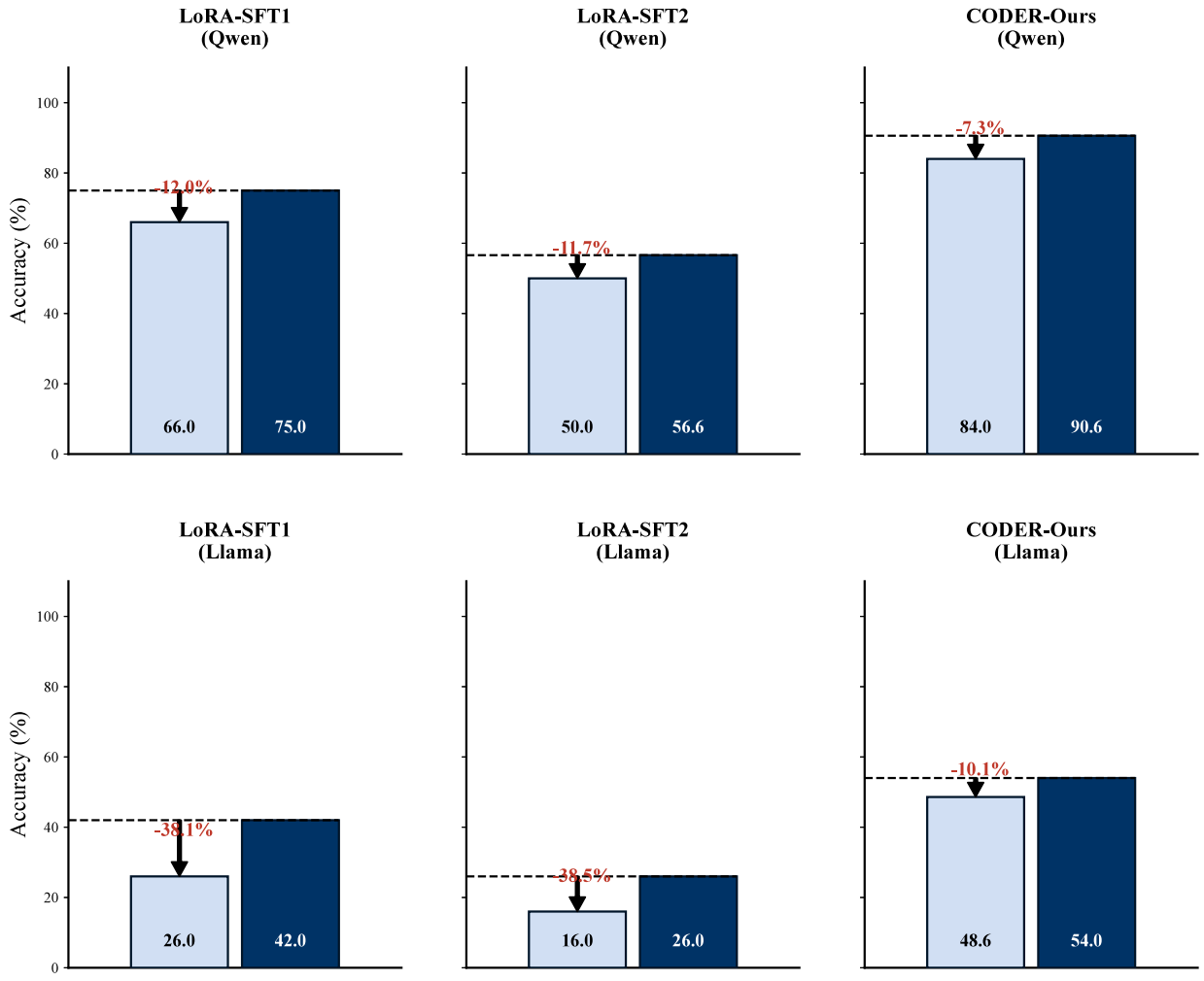


Fig. 15. Resilience against semantic distractors across different architectures. The top row shows results for Qwen2.5-Math-7B, and the bottom row shows Llama-3.1-8B. The light blue bars represent the performance under distractor-injected scenarios, with red arrows indicating the accuracy degradation relative to clean benchmarks (dark blue bars).

Table 8
Ablation study on data distribution ratios. P-I-A refers to the proportion of primary, intermediate, and advanced samples. The pyramid distribution (Ours) achieves the best balance between basic and advanced reasoning tasks. For AMC 23, results are reported as P@1 / P@16.

Base Model	Configuration	Distribution (P-I-A)	GSM8K (Basic)	AMC 23 (P@1 / P@16)
Qwen	**Ours (Pyramid)**	**30% – 50% – 20%**	**90.6±0.3**	**40.0 / 82.9**
	Uniform	33% – 33% – 33%	88.2±0.5	36.4 / 75.4
	Bottom-Heavy	60% – 30% – 10%	90.1±0.4	29.8 / 61.8
	Top-Heavy	10% – 30% – 60%	81.5±0.7	38.7 / 80.2
Llama	**Ours (Pyramid)**	**30% – 50% – 20%**	**48.6±0.4**	**22.5 / 48.0**
	Uniform	33% – 33% – 33%	42.1±0.7	19.5 / 41.5
	Bottom-Heavy	60% – 30% – 10%	46.8±0.6	15.3 / 32.7
	Top-Heavy	10% – 30% – 60%	35.4±0.9	21.2 / 45.3

4.8. Robustness analysis: resilience to contextual distractors

A critical hallmark of "deep reasoning" is the capability to maintain logical integrity while filtering out irrelevant contextual information. As recently highlighted by Wu et al. (2025), robust mathematical solvers must possess mechanisms to prune semantic distractors that do not contribute to the core reasoning chain. To evaluate whether the manifold smoothing induced by NEFTune and our difficulty-aware core-set selection fosters such resilience, we conducted a Distractor-Injected Stress Test.

Experimental Design. We randomly sampled 200 questions from the GSM8K test set and constructed a Distracted version by injecting semantically plausible but logically irrelevant suffixes (e.g., tangential facts about geography or weather). We then performed a head-to-head comparison between the *Clean* and *Distracted* scenarios across both Qwen2.5-Math-7B and Llama-3.1-8B architectures.

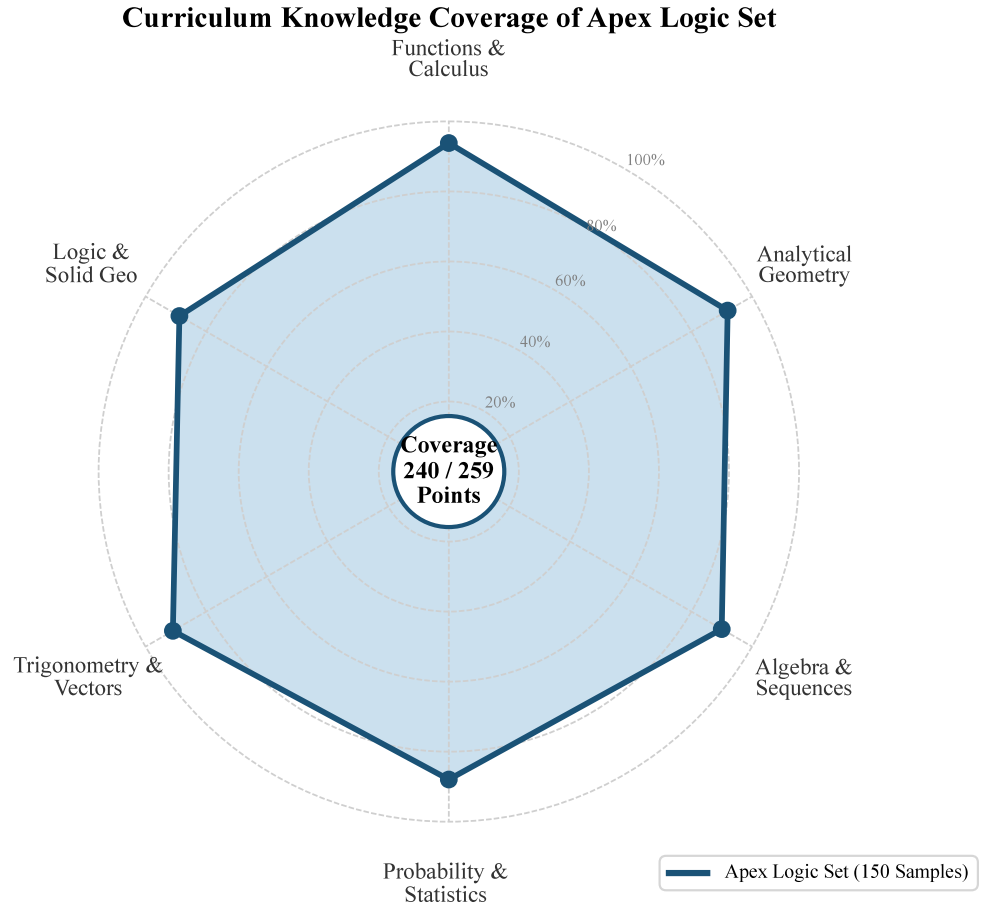


Fig. 16. Curriculum coverage of the Math-Core-700 dataset.

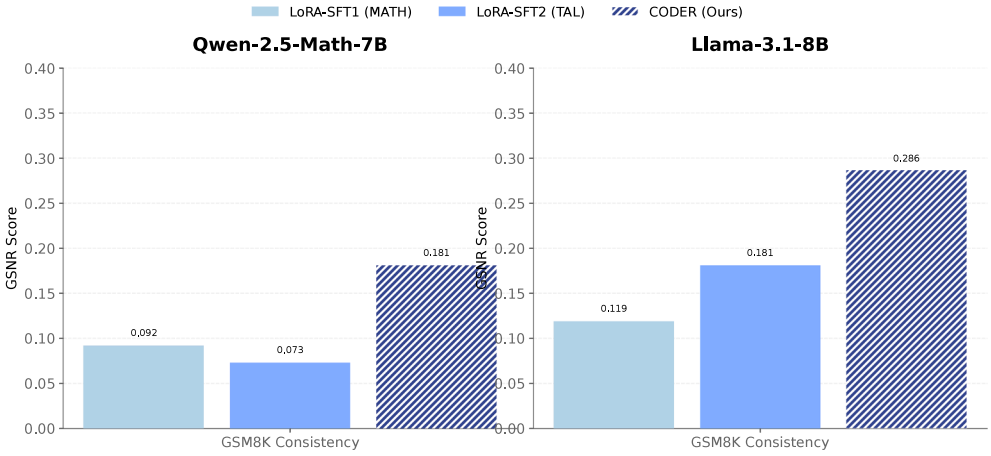


Fig. 17. GSNR scores across 50 diverse mathematical reasoning tasks.

Quantitative Results and Logical Elasticity. The experimental results, illustrated in Fig. 15, reveal a notable contrast in robustness between CODER and the LoRA baselines:

- **General-purpose Llama-3.1-8B:** The LoRA-SFT baselines exhibited substantial performance degradation, with drops of 38.1% and 38.5%, respectively. In contrast, CODER showed greater resilience, with a degradation of only 10.1%.
- **Math-specialized Qwen2.5-Math-7B:** While the specialized baselines showed better baseline stability, they still experienced drops of approximately 12%. CODER maintained a more stable performance, with a decrease of 7.3%.

Observational Interpretation: Logic vs. Surface Noise. We observe that this resilience is consistent with the nature of the weight updates and the manifold smoothing effect. Qualitative analysis of CODER's reasoning chains suggests that the model's internal ⟨think⟩ process tends to anchor its derivation onto the essential components of the problem, which may help prune irrelevant paths. While distractors may introduce surface-level noise to the output distribution, the core mathemati-

cal logic within the latent reasoning manifold appears to remain largely intact.

This observation is consistent with the interpretation that by prioritizing high-density logical primitives during training, CODER may develop a more robust reasoning structure capable of distinguishing between conversational surface and logical core. This capability aligns with the requirement for distractor-resilient reasoning (Wu et al., 2025), and is consistent with the view that CODER's performance may benefit from the acquisition of robust logical patterns rather than relying on surface-level associations.

4.9. Core-set design validation: distribution, coverage, and signal coherence

The efficiency of CODER raises a fundamental question: why can a compact set of 700 samples, structured in a 30%-50%-20% pyramid, outperform datasets orders of magnitude larger? To address this, we conduct an ablation study on data distribution and a curriculum-based mapping of our knowledge coverage.

Pedagogical Scaffolding and the 30%-50%-20% Ratio. Our choice of the Difficulty Pyramid draws motivation from the *Scaffolding Theory* of educational psychology. As shown in Table 8, we compared our pyramid distribution against three representative variants: *Uniform*, *Bottom-Heavy* (emphasizing foundations), and *Top-Heavy* (emphasizing competition-level problems). The results reveal a clear pattern: while the *Bottom-Heavy* configuration maintains high performance on basic tasks (GSM8K: 90.1% for Qwen), it shows a notable drop on advanced reasoning (AMC 23 P@16: 61.8%), falling substantially behind the pyramid distribution (82.9%). Conversely, the *Top-Heavy* approach, while maintaining competitive reasoning on AMC 23 (80.2% for Qwen), exhibits degraded foundational performance on GSM8K (81.5%), suggesting a disconnect where the model struggles with foundational consistency. The 30%-50%-20% ratio offers a balanced structure: the 30% primary samples support basic logical primitives, the 50% intermediate samples build multi-step reasoning chains, and the 20% apex samples provide exposure to high-order abstraction. This balanced distribution may facilitate the formation of a more continuous reasoning manifold, potentially reducing over-reliance on isolated pattern-matching rules.

Logical Hubs: Assessing the Sufficiency of Apex Samples. A key concern is whether close to 150 apex samples (20% of our set) are sufficient to cover the vast landscape of high-school mathematics. To quantify this, we mapped our Apex Logic Set to the National Curriculum Standards across six core domains. As illustrated in the radar chart (Fig. 16), our compact set covers 240 out of 259 total knowledge points (92.7% coverage). This outcome—achieving substantial coverage with few samples—can be attributed to the interconnected nature of advanced mathematical problems. Unlike primary arithmetic, a single apex-level problem (e.g., an analytical geometry proof) simultaneously engages multiple concepts such as algebraic manipulation, geometric properties, and logical deduction. By selecting samples with high difficulty centrality, each apex problem serves as a dense information carrier, effectively covering the curriculum's breadth through logical synthesis rather than repetition.

To better understand the factors contributing to CODER's effectiveness, we conduct a post-hoc Gradient Signal-to-Noise Ratio (GSNR) analysis across 50 diverse mathematical reasoning tasks. GSNR, defined as the ratio of the squared mean gradient to the gradient variance, serves as a quantitative proxy for the coherence of the learning signal; a higher ratio suggests that the model may have acquired more coherent and generalizable logical patterns across different problems. As illustrated in Fig. 17, CODER consistently yields higher GSNR scores than large-scale LoRA baselines across both the math-specialized Qwen2.5-Math-7B and the general-purpose Llama-3.1-8B architectures. Specifically, CODER achieves a 140% consistency gain in Llama and a 97% improvement in Qwen compared to their respective MATH-LoRA baselines. This cross-architectural pattern offers empirical support consistent with the interpretation that CODER may facilitate a more aligned optimization trajectory.

This gap in the GSNR profile is consistent with the interpretation that a 700-sample difficulty-aware core set can be competitive with training sets an order of magnitude larger. The elevated GSNR in CODER suggests that our data selection may identify high-density logical primitives that evoke consistent directional updates in the parameter space, potentially reducing the "gradient confusion" often associated with redundant or noisy large-scale data. Moreover, this pattern is consistent with the view that DoRA and NEFTune act synergistically to encourage convergence toward a relatively flat minimum on the loss landscape. A flatter landscape is generally associated with improved robustness to surface-level input variations, which aligns with the observed resilience of CODER to semantic distractors and its generalization on unseen competition-level problems.

Human Validation of Distilled Reasoning Chains. To ensure that the performance gains are driven by superior logic rather than data artifacts, we conducted a blind human evaluation of our reconstructed thought chains. Specifically, three senior mathematics undergraduate and graduate students served as expert annotators to independently evaluate 100 randomly sampled CODER-generated responses.

The evaluation was conducted across three distinct dimensions on a 5-point Likert scale: *Logical Correctness*, *Step-wise Coherence*, and *Distractor Resilience*. To assess evaluation reliability, we calculated the inter-annotator agreement, achieving a Fleiss' κ of 0.82, which indicates strong consensus among annotators. Human evaluation yielded an average score of 4.85/5.0, indicating that the GPT-5-driven dual-track reconstruction produces high-quality reasoning chains.

5. Conclusions and future directions

This study addresses the challenge of activating sophisticated mathematical reasoning within resource-constrained environments by introducing the **CODER** framework. Through the synergy of an empirical, difficulty-stratified *Math-Core-700* curriculum and a collaborative regularization strategy involving DoRA and NEFTune, we show that competitive performance on benchmarks such as Gaokao-Bench and AMC 23 can be achieved on a single consumer-grade GPU using a small fraction of the data volume typically employed in comparable recipes. Our findings are consistent with the view that coupling high Signal-to-Noise Ratio (SNR) data with decoupled optimization may effectively facilitate the reconstruction of the model's inference manifold, potentially offering a pathway to mitigate the diminishing returns associated with massive data scaling. We emphasize that these findings are correlational and should be interpreted as empirical evidence of effective activation, rather than as a causal proof of the underlying reasoning mechanisms.Ultimately, CODER provides a feasible blueprint for Green AI and the democratization of high-performance large model training, reducing the reliance on prohibitive computational costs.

While CODER demonstrates notable efficiency in activating reasoning capabilities through its 700-sample difficulty-aware core-set, its effectiveness is closely tied to the quality of the foundation model's pre-training, acting primarily as an *in-situ* activator of latent knowledge rather than an original knowledge source. Several limitations warrant acknowledgment. First, the reasoning chains used for training are distilled from a single teacher model (GPT-5), which may introduce bias and limit the diversity of reasoning strategies; exploring multi-teacher distillation or self-consistency verification could be a promising direction to mitigate this issue. Second, the limited sample size inherent in this few-shot paradigm may encounter bottlenecks regarding long-tail knowledge coverage and rigid adherence to complex output formats. Third, while our current empirical validation focuses on 7B/8B dense models, the applicability of CODER to ultra-large parameters or Mixture-of-Experts (MoE) architectures remains to be verified. To explore these directions, future work could investigate preference-guided alignment—such as DPO-based low-resource optimization—to further

refine logical rigor and reduce residual hallucinations, while exploring automated curriculum learning and multimodal reasoning to develop more robust, generalizable, and computationally sustainable intelligent systems in resource-constrained environments.

CRediT authorship contribution statement

Lei Wang: Validation, Supervision, Methodology, Formal analysis, Data curation, Conceptualization; **GuoAo Liu:** Software, Methodology, Formal analysis, Data curation, Conceptualization; **XiaoYong Zhao:** Supervision.

Funding

Project Title: Research and Application of Intelligent Agents for Ecological Environment Smart Management

Project Number: 2025ZD1206201

Major Program: National Key Science and Technology Program for Comprehensive Environmental Governance in the Beijing-Tianjin-Hebei Region

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: GuoAo Liu reports was provided by Information Center of the Ministry of Ecology and Environment. GuoAo Liu reports a relationship with Information Center of the Ministry of Ecology and Environment that includes: funding grants. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Prompt engineering for chain-of-thought refinement

To transform raw mathematical problems into training samples incorporating high-density reasoning processes, this study designed a prompt engineering framework based on a teacher model (GPT-5). This framework aims to induce the teacher model, through explicit instruction constraints, to enter a latent variable space, denoted as ⟨think⟩, before generating the final answer. Within this space, the model is prompted to execute Dual-Track Reasoning: simultaneously conducting forward logical deduction while undertaking reverse self-critique and trial-and-error.

The core System Prompt and User Prompt used for data reconstruction are presented below.

Teacher Model Specifications and Data Reconstruction. The structural reconstruction of the Math-Core-700 dataset was conducted using the OpenAI GPT-5 API (specifically the `gpt-5-2025-08-08` snapshot). To ensure the high-density logical rigor and reproducibility of the distilled reasoning chains, we employed a controlled decoding configuration: a temperature of 0.2, a `top_p` of 0.95, and a frequency penalty of 0.0. These parameters were selected to prioritize logical consistency over creative variance. The data synthesis process was carried out between October 2025 and January 2026. The reconstruction process utilized a "Dual-Track" prompting strategy, which is detailed alongside the complete system prompts in Appendix A. This exhaustive disclosure ensures that our data-centric fine-tuning paradigm can be systematically replicated by the research community using the same publicly available API endpoints.

A.1. System prompt design

System directives not only define the role of the teacher model but also strictly prescribe the output format specifications (LaTeX) and cognitive behavioral patterns (self-reflection). The core directives are as follows:

Role Definition: You are an expert professor proficient in mathematical principles and problem-solving strategies. Your task is to provide comprehensive, rigorous, and logically self-contained solutions for the given mathematical problems.

1. Explicit Reasoning Stream: Before presenting your final answer, you must encapsulate your thought process within <think> and </think> tags.
2. Reflection & Trial-and-Error: Within the <think> tags, you should not merely list steps but demonstrate the "dynamic process" of your reasoning. This includes:Identifying implicit conditions: Analyzing geometric properties or algebraic constraints not explicitly stated in the question but crucial for solving it.Strategy evaluation: Proposing at least two possible solution paths, assessing their feasibility, and explaining why the current path was chosen.If computational errors or logical dead ends are discovered, explicitly self-correct within the tags.
3. Formatting conventions: All mathematical symbols, variables, and formulas must adhere to standard LaTeX format (e.g., use x rather than x, and $\frac{a}{b}$ rather than a/b).
4. Output structure: Upon concluding your thought process, present the final standardized solution (Solution) and answer (Answer) outside the <think> tag.

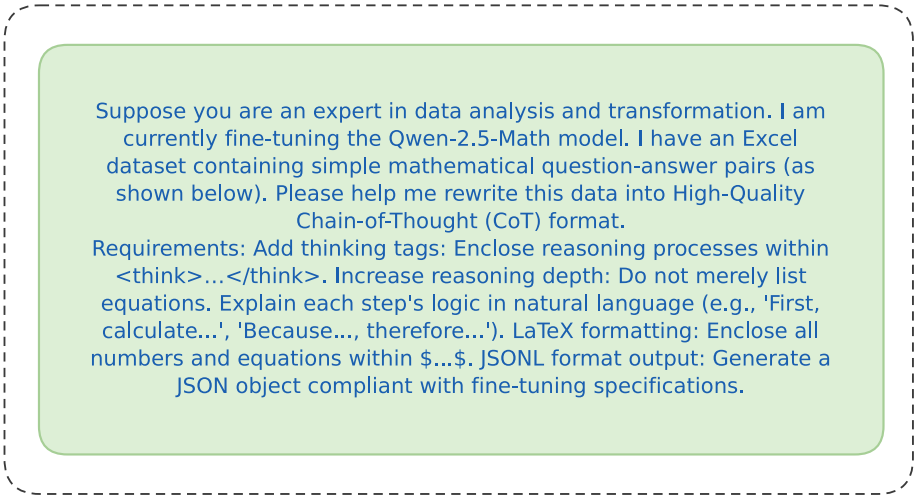


Fig. A.1. Examples of prompts for reconstructing thought chains.

A.2. User prompt template

To ensure the model remains focused on the current mathematical problem, we have adopted the following input template:
Input question: {Original_Question}
Instruction: Please reconstruct the complete chain of reasoning for this problem in accordance with the aforementioned system requirements. Note that your reasoning should be as thorough as a textbook, explaining clearly why each step is taken, not merely how it is done. Begin your reasoning (see Fig. A.1):

A.3. Quality control and cleaning mechanisms

Based on the raw data generated from the aforementioned prompts, we implemented post-processing cleansing algorithms to ensure data quality:

1. Format validation: Utilize regular expressions to extract text within <think> tags, detecting LaTeX syntax errors or unclosed brackets.
2. Consistency filtering: Automatically compare inferences within <think> tags against the final output Answer for consistency. Samples exhibiting logical discontinuities or hallucinations (where the reasoning process is correct but the answer is erroneous, or vice versa) are discarded.

Appendix B. Comprehensive test results

B.1. Reporting protocol

All fine-tuning experiments on the non-competition benchmarks were repeated with three independent random seeds (42, 123, and 2024). For each seed, a separate model was trained and the final checkpoint after exactly 5 training epochs was evaluated. No validation-based early stopping or checkpoint selection was applied, so that all methods are compared under the same checkpoint-selection rule.

For non-competition benchmarks (GSM8K, CMATH, Gaokao-Math, C-Eval, and CCEE-CN), we report the mean and standard deviation across the three seeds, together with per-seed results, 95% confidence intervals, and paired significance tests. The 95% confidence interval is computed as

$$\mathrm{CI}_{95\%} = \bar{x} \pm t_{0.975,2} \frac{s}{\sqrt{3}},$$

where $t_{0.975,2} = 4.303$. Paired significance tests compare CODER against each baseline using the same seed pairing (42 with 42, 123 with 123, 2024 with 2024). With only three seeds, these paired t-tests are exploratory and should be interpreted alongside the per-seed results and confidence intervals.

For the high-difficulty competition benchmarks (AMC 23, CNMO-CN, and Hard-CN), results were obtained from a single training seed and evaluated as Pass@1 / Pass@16 under a matched sampling budget. Because they rest on a single seed, no across-seed deviation is defined for them, and they are therefore not part of the per-seed analysis, confidence intervals or paired tests reported above.

B.2. Per-seed results on Qwen2.5-Math-7B

See Table B.1.

Table B.1
Per-seed accuracy (%) on Qwen2.5-Math-7B for non-competition benchmarks. Seeds are 42, 123, and 2024.

Benchmark	Method	Seed 42	Seed 123	Seed 2024	Mean ± SD
GSM8K	Base	90.3	89.6	90.1	90.0 ± 0.4
GSM8K	SFT1	75.5	74.4	75.1	75.0 ± 0.6
GSM8K	SFT2	57.8	55.5	56.5	56.6 ± 1.1
GSM8K	**CODER**	90.3	90.9	90.6	**90.6 ± 0.3**
CMATH	Base	84.6	85.6	84.8	85.0 ± 0.5
CMATH	SFT1	81.7	83.8	83.5	83.0 ± 1.1
CMATH	SFT2	90.5	90.1	89.4	90.0 ± 0.6
CMATH	**CODER**	93.9	93.3	93.6	**93.6 ± 0.3**
Gaokao-Math	Base	56.9	57.4	58.5	57.6 ± 0.8
Gaokao-Math	SFT1	43.4	40.7	41.9	42.0 ± 1.3
Gaokao-Math	SFT2	59.3	61.0	59.7	60.0 ± 0.9
Gaokao-Math	**CODER**	79.4	78.6	79.0	**79.0 ± 0.4**
C-Eval Adv. Math	Base	32.4	31.0	31.1	31.5 ± 0.8
C-Eval Adv. Math	SFT1	37.9	35.6	36.9	36.8 ± 1.2
C-Eval Adv. Math	SFT2	26.9	24.7	27.3	26.3 ± 1.4
C-Eval Adv. Math	**CODER**	47.6	47.0	47.3	**47.3 ± 0.3**
C-Eval Middle School	Base	84.5	84.7	83.4	84.2 ± 0.7
C-Eval Middle School	SFT1	94.2	95.1	93.9	**94.4 ± 0.6**
C-Eval Middle School	SFT2	84.5	83.4	82.0	83.3 ± 1.3
C-Eval Middle School	**CODER**	89.5	88.9	88.6	89.0 ± 0.5
C-Eval High School	Base	58.0	59.7	58.7	58.8 ± 0.9
C-Eval High School	SFT1	44.8	45.5	42.9	44.4 ± 1.3
C-Eval High School	SFT2	43.1	46.1	44.0	44.4 ± 1.5
C-Eval High School	**CODER**	72.4	71.6	72.0	**72.0 ± 0.4**
C-Eval Prob. & Stat.	Base	51.0	50.2	48.8	50.0 ± 1.1
C-Eval Prob. & Stat.	SFT1	54.3	55.6	56.6	55.5 ± 1.2
C-Eval Prob. & Stat.	SFT2	47.3	45.5	48.2	47.0 ± 1.4
C-Eval Prob. & Stat.	**CODER**	67.2	66.2	66.7	**66.7 ± 0.5**
CCEE-CN	Base	50.8	51.0	50.0	50.6 ± 0.5
CCEE-CN	SFT1	53.7	53.1	52.8	53.2 ± 0.5
CCEE-CN	SFT2	41.8	40.7	42.0	41.5 ± 0.7
CCEE-CN	**CODER**	70.8	70.0	70.4	**70.4 ± 0.4**

B.3. Per-seed results on Llama-3.1-8B

See Table B.2.

Table B.2
Per-seed accuracy (%) on Llama-3.1-8B for non-competition benchmarks. Seeds are 42, 123, and 2024.

Benchmark	Method	Seed 42	Seed 123	Seed 2024	Mean ± SD
GSM8K	Base	46.0	45.0	45.5	45.5 ± 0.5
GSM8K	SFT1	28.7	30.0	28.6	29.1 ± 0.8
GSM8K	SFT2	16.6	18.5	16.2	17.1 ± 1.2
GSM8K	**CODER**	49.0	48.2	48.6	**48.6 ± 0.4**
CMATH	Base	0.0	0.0	0.0	0.0 ± 0.0
CMATH	SFT1	23.6	24.6	22.3	23.5 ± 1.2
CMATH	SFT2	21.5	22.1	19.4	21.0 ± 1.4
CMATH	**CODER**	49.2	47.8	48.5	**48.5 ± 0.7**
Gaokao-Math	Base	2.8	2.0	2.7	2.5 ± 0.4
Gaokao-Math	SFT1	4.7	6.7	6.6	6.0 ± 1.1
Gaokao-Math	SFT2	9.6	8.3	7.3	8.4 ± 1.2
Gaokao-Math	**CODER**	20.6	19.4	20.0	**20.0 ± 0.6**
C-Eval Adv. Math	Base	0.0	0.0	0.0	0.0 ± 0.0
C-Eval Adv. Math	SFT1	4.1	7.1	5.3	5.5 ± 1.5
C-Eval Adv. Math	SFT2	11.8	9.3	10.4	10.5 ± 1.3
C-Eval Adv. Math	**CODER**	17.3	15.9	16.6	**16.6 ± 0.7**
C-Eval Middle School	Base	0.0	0.0	0.0	0.0 ± 0.0
C-Eval Middle School	SFT1	11.8	11.0	8.7	10.5 ± 1.6
C-Eval Middle School	SFT2	19.9	20.6	22.8	21.1 ± 1.5
C-Eval Middle School	**CODER**	28.5	26.9	27.7	**27.7 ± 0.8**
C-Eval High School	Base	0.0	0.0	0.0	0.0 ± 0.0
C-Eval High School	SFT1	9.4	10.0	12.1	10.5 ± 1.4
C-Eval High School	SFT2	26.6	27.5	29.3	27.8 ± 1.4
C-Eval High School	**CODER**	22.7	22.1	21.8	**22.2 ± 0.5**
C-Eval Prob. & Stat.	Base	0.0	0.0	0.0	0.0 ± 0.0
C-Eval Prob. & Stat.	SFT1	18.4	15.7	16.0	16.7 ± 1.5
C-Eval Prob. & Stat.	SFT2	9.7	10.7	12.9	11.1 ± 1.6
C-Eval Prob. & Stat.	**CODER**	28.3	27.1	27.7	**27.7 ± 0.6**
CCEE-CN	Base	2.3	2.2	2.1	2.2 ± 0.1
CCEE-CN	SFT1	6.8	6.7	6.9	6.8 ± 0.1
CCEE-CN	SFT2	9.0	9.1	9.2	9.1 ± 0.1
CCEE-CN	**CODER**	30.0	29.0	29.5	**29.5 ± 0.5**

B.4. 95% confidence intervals for CODER

See Table B.3.

Table B.3
Mean, standard deviation, and 95% confidence interval for CODER across three independent seeds on non-competition benchmarks. The interval is computed as $\bar{x} \pm t_{0.975,2}\, s/\sqrt{3}$, with $t_{0.975,2} = 4.303$.

Model	Benchmark	Mean ± SD	95% CI
Qwen	GSM8K	90.60 ± 0.30	[89.85, 91.35]
Qwen	CMATH	93.60 ± 0.30	[92.85, 94.35]
Qwen	Gaokao-Math	79.00 ± 0.40	[78.01, 79.99]
Qwen	C-Eval Adv. Math	47.30 ± 0.30	[46.55, 48.05]
Qwen	C-Eval Middle School	89.00 ± 0.5	[87.86, 90.14]
Qwen	C-Eval High School	72.00 ± 0.40	[71.01, 72.99]
Qwen	C-Eval Prob. & Stat.	66.70 ± 0.50	[65.46, 67.94]
Qwen	CCEE-CN	70.40 ± 0.40	[69.41, 71.39]
Llama	GSM8K	48.60 ± 0.40	[47.61, 49.59]
Llama	CMATH	48.50 ± 0.70	[46.76, 50.24]
Llama	Gaokao-Math	20.00 ± 0.60	[18.51, 21.49]
Llama	C-Eval Adv. Math	16.60 ± 0.70	[14.86, 18.34]
Llama	C-Eval Middle School	27.70 ± 0.80	[25.71, 29.69]
Llama	C-Eval High School	22.20 ± 0.5	[21.06, 23.34]
Llama	C-Eval Prob. & Stat.	27.70 ± 0.60	[26.21, 29.19]
Llama	CCEE-CN	29.50 ± 0.50	[28.26, 30.74]

B.5. Paired significance tests on Qwen2.5-Math-7B

See Table B.4.

Table B.4
Paired t-tests on Qwen2.5-Math-7B across three seeds for non-competition benchmarks. Each comparison uses the same seed pairing (42 with 42, 123 with 123, 2024 with 2024). With only three seeds, these tests are exploratory and should be interpreted alongside the per-seed results and confidence intervals.

Benchmark	Comparison	Mean diff. Δ	$t(2)$	p
GSM8K	CODER − Base	+0.6	1.58	0.254
GSM8K	CODER − SFT1	+15.6	31.62	0.0010
GSM8K	CODER − SFT2	+34.0	40.54	0.00061
CMATH	CODER − Base	+8.6	18.20	0.0030
CMATH	CODER − SFT1	+10.6	12.95	0.0059
CMATH	CODER − SFT2	+3.6	11.78	0.0071
Gaokao-Math	CODER − Base	+21.4	36.52	0.00075
Gaokao-Math	CODER − SFT1	+37.0	67.18	0.00022
Gaokao-Math	CODER − SFT2	+19.0	25.78	0.0015
C-Eval Adv.	CODER − Base	+15.8	51.72	0.00037
C-Eval Adv.	CODER − SFT1	+10.5	21.29	0.0022
C-Eval Adv.	CODER − SFT2	+21.0	30.85	0.0010
C-Eval Middle	CODER − Base	+4.8	15.71	0.0040
C-Eval Middle	CODER − SFT1	−5.4	−12.39	0.0065
C-Eval Middle	CODER − SFT2	+5.7	12.06	0.0068
C-Eval High	CODER − Base	+13.2	18.25	0.0030
C-Eval High	CODER − SFT1	+27.6	31.87	0.0010
C-Eval High	CODER − SFT2	+27.6	24.75	0.0016
C-Eval Prob./Stat.	CODER − Base	+16.7	27.71	0.0013
C-Eval Prob./Stat.	CODER − SFT1	+11.2	12.99	0.0059
C-Eval Prob./Stat.	CODER − SFT2	+19.7	30.64	0.0011
CCEE-CN	CODER − Base	+19.8	47.56	0.00044
CCEE-CN	CODER − SFT1	+17.2	82.63	0.00015
CCEE-CN	CODER − SFT2	+28.9	109.23	< 0.0001

B.6. Paired significance tests on Llama-3.1-8B

See Table B.5.

Table B.5
Paired t-tests on Llama-3.1-8B across three seeds for non-competition benchmarks. Each comparison uses the same seed pairing (42 with 42, 123 with 123, 2024 with 2024). With only three seeds, these tests are exploratory and should be interpreted alongside the per-seed results and confidence intervals.

Benchmark	Comparison	Mean diff. Δ	$t(2)$	p
GSM8K	CODER − Base	+3.1	53.69	0.00035
GSM8K	CODER − SFT1	+19.5	29.74	0.0011
GSM8K	CODER − SFT2	+31.5	35.00	0.00082
CMATH	CODER − Base	+48.5	120.01	< 0.0001
CMATH	CODER − SFT1	+25.0	27.28	0.0013
CMATH	CODER − SFT2	+27.5	27.87	0.0013
Gaokao-Math	CODER − Base	+17.5	114.56	< 0.0001
Gaokao-Math	CODER − SFT1	+14.0	14.41	0.0048
Gaokao-Math	CODER − SFT2	+11.6	21.06	0.0022
C-Eval Adv.	CODER − Base	+16.6	41.07	0.00059
C-Eval Adv.	CODER − SFT1	+11.1	8.71	0.0129
C-Eval Adv.	CODER − SFT2	+6.1	18.98	0.0028
C-Eval Middle	CODER − Base	+27.7	59.97	0.00028
C-Eval Middle	CODER − SFT1	+17.2	18.51	0.0029
C-Eval Middle	CODER − SFT2	+6.6	6.12	0.0257
C-Eval High	CODER − Base	+22.2	83.91	0.00014
C-Eval High	CODER − SFT1	+11.7	11.06	0.0081
C-Eval High	CODER − SFT2	−5.6	−5.36	0.0330
C-Eval Prob./Stat.	CODER − Base	+27.7	79.96	0.00016
C-Eval Prob./Stat.	CODER − SFT1	+11.0	19.76	0.0026
C-Eval Prob./Stat.	CODER − SFT2	+16.6	15.07	0.0044
CCEE-CN	CODER − Base	+27.3	103.18	< 0.0001
CCEE-CN	CODER − SFT1	+22.7	85.80	0.00014
CCEE-CN	CODER − SFT2	+20.4	63.46	0.00025

Appendix C. Token statistics of Math-Core-700

We report the length statistics of the Math-Core-700 training corpus. Because the two backbones use different tokenizers, and because the same text is segmented differently by each, the counts below are produced separately with the tokenizer of the corresponding backbone. For every instance we report the number of tokens in (i) the problem statement, (ii) the reconstructed thinking chain, (iii) the final answer, (iv) the response as a whole, i.e. the thinking chain plus the final answer, and (v) the complete training instance as presented to the model. The table reports measured values throughout; no component figures are derived, so the complete-instance totals should not be expected to equal the sum of the reported components exactly. Under both tokenizers the response carries the large majority of the tokens (88.1% and 87.2% of the problem-plus-response total, respectively), and the thinking chain alone accounts for 68.2% and 67.1%. Under the assistant-only objective of Section 3.2.4, the supervision signal is therefore concentrated almost entirely in the reconstructed reasoning chains rather than in the problem statements.

C.1. Qwen2.5 tokenizer

See Table C.1.

Table C.1
Token statistics of Math-Core-700 under the Qwen2.5 tokenizer. All values are token counts. “P95” denotes the 95th percentile.

Component	Mean	Median	P95	Max	Sum
Problem statement	50.8	48.0	90.9	143	35,647
Thinking chain	292.2	169.5	953.3	3228	205,147
Final answer	81.4	20.0	397.4	1281	57,117
Response (total)	377.6	195.0	1,329.9	3969	265,072
Complete instance	458.4	275.0	1,439.7	4068	321,779

C.2. lLama-3.1 tokenizer

See Table C.2.

Table C.2
Token statistics of Math-Core-700 under the Llama-3.1 tokenizer. All values are token counts. “P95” denotes the 95th percentile.

Component	Mean	Median	P95	Max	Sum
Problem statement	54.6	52.0	96.9	161	38,332
Thinking chain	285.6	157.0	939.6	3125	200,499
Final answer	81.6	20.0	398.3	1317	57,303
Response (total)	371.2	185.0	1,323.3	3869	260,610
Complete instance	425.3	240.0	1,392.7	3935	298,557

C.3. Sequence-length distribution

See Table C.3.

Table C.3
Distribution of complete-instance lengths in Math-Core-700, expressed as the fraction of instances exceeding each threshold, under the Qwen2.5 and Llama-3.1 tokenizers respectively.

Sequence length	Qwen2.5	Llama-3.1
> 512 tokens	23.8%	21.9%
> 1024 tokens	10.0%	9.5%
> 2048 tokens	2.0%	1.9%
> 4096 tokens	0.0%	0.0%

References

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). Training verifiers to solve math word problems. arXiv:2110.14168.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., & et al. (2024). The Llama 3 herd of models. arXiv:2407.21783.

Dziri, N., Lu, X., Sclar, M., Li, X. L., Jiang, L., Lin, B. Y., Welleck, S., West, P., Bhagavatula, C., Le Bras, R., Hwang, J., Sanyal, S., Ren, X., Ettinger, A., Harchaoui, Z., & Choi, Y. (2023). Faith and fate: Limits of transformers on compositionality. In *Advances in neural information processing systems* (pp. 70293–70332). (*vol. 36*).

Han, Z., Gao, C., Liu, J., Zhang, J., & Zhang, S. Q. (2024). Parameter-efficient fine-tuning with discrete Fourier transform. arXiv:2405.03003.

Guan, X., Zhang, L. L., Liu, Y., Shang, N., Sun, Y., Zhu, Y., Yang, F., & Yang, M. (2025). rStar-Math: Small LLMs can master math reasoning with self-evolved deep thinking. arXiv:2501.04519.

Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., & et al. (2025). DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International conference on learning representations*.

Huang, Y., Bai, Y., Zhu, Z., Zhang, J., Zhang, J., Su, T., Liu, J., Lv, C., Zhang, Y., Lei, J., Fu, Y., Sun, M., & He, J. (2023). C-Eval: A multi-level multi-discipline Chinese evaluation suite for foundation models. In *Advances in neural information processing systems* (pp. 62991–63010). (*vol. 36*).

Jain, N., Chiang, P.-Y., Wen, Y., Kirchenbauer, J., Chu, H.-M., Somepalli, G., Bartoldson, B. R., Kailkhura, B., & et al. (2023). NEFTune: Noisy embeddings improve instruction finetuning. arXiv:2310.05914.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. arXiv:2001.08361.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., & Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, *114*(13), 3521–3526.

Kojima, T., Gu, S. (S.), Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large language models are zero-shot reasoners. In *Advances in neural information processing systems* (pp. 22199–22213). (*vol. 35*).

Lewkowycz, A., Andreassen, A., Dohan, D., Dyer, E., Michalewski, H., Ramasesh, V., Slone, A., Anil, C., Schlag, I., Gutman-Solo, T., Wu, Y., Neyshabur, B., Gur-Ari, G., & Misra, V. (2022). Solving quantitative reasoning problems with language models. In *Advances in neural information processing systems* (pp. 3843–3857). (*vol. 35*).

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2023). Let’s verify step by step. In *The twelfth international conference on learning representations*.

Liu, W., Hu, H., Zhou, J., Ding, Y., Li, J., Zeng, J., He, M., Chen, Q., Jiang, B., Zhou, A., & He, L. (2025). Are your LLMs capable of stable reasoning? In *Findings of the association for computational linguistics: ACL 2025*.

Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., & Chen, M.-H. (2024). DoRA: Weight-decomposed low-rank adaptation. In *International conference on machine learning*.

Luo, H., Sun, Q., Xu, C., Zhao, P., Lou, J., Tao, C., Geng, X., Lin, Q., Chen, S., & Zhang, D. (2023). WizardMath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. arXiv:2308.09583.

Meng, F., Wang, Z., & Zhang, M. (2024). PiSSA: Principal singular values and singular vectors adaptation of large language models. In *Advances in neural information processing systems* (pp. 121038–121072). (*vol. 37*).

OpenAI (2023). GPT-4 technical report. arXiv:2303.08774.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300.

Tang, Z., Zhang, X., Wang, B., & Wei, F. (2024). MathScale: Scaling instruction tuning for mathematical reasoning. arXiv:2403.02884.

Toshniwal, S., Du, W., Moshkov, I., Kisacanin, B., Ayrapetyan, A., & Gitman, I. (2024). OpenMathInstruct-2: Accelerating AI for math with massive open-source instruction data. arXiv:2410.01560.

Wang, P., Li, L., Shao, Z., Xu, R., Dai, D., Li, Y., Chen, D., Wu, Y., & Sui, Z. (2024a). Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*.

Wang, Z., Liang, J., He, R., Wang, Z., & Tan, T. (2024b). LoRA-Pro: Are low-rank adapters properly optimized? arXiv:2407.18242.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in neural information processing systems* (pp. 24824–24837). (*vol. 35*).

Wei, T., Luan, J., Liu, W., Dong, S., & Wang, B. (2023). CMath: Can your language model pass Chinese elementary school math test? arXiv:2306.16636.

Wu, S., Jin, Y.-F., Niu, P., Datta, A., & Andersson, S. B. (2025). Robust knowledge editing via explicit reasoning chains for distractor-resilient multi-hop QA. arXiv:2509.01468.

Xu, Z., Jiang, F., Niu, L., Deng, Y., Poovendran, R., Choi, Y., & Lin, B. Y. (2024). Magpie: Alignment data synthesis from scratch by prompting aligned LLMs with nothing. arXiv:2406.08464.

Yang, A., Zhang, B., Hui, B., Gao, B., Yu, B., Li, C., Liu, D., Tu, J., Zhou, J., Lin, J., Lu, K., Xue, M., Ren, X., & Zhang, Z. (2024). Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. arXiv:2409.12122.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., & et al. (2025). Qwen3 technical report. arXiv:2505.09388.

Ye, Y., Huang, Z., Xiao, Y., Chern, E., Xia, S., & Liu, P. (2025). LIMO: Less is more for reasoning. arXiv:2502.03387.

Zhong, Q., Ding, L., Liao, F., Liu, J., Du, B., & Tao, D. (2026). Resolving knowledge conflicts in domain-specific data selection: A case study on medical instruction-tuning. *IEEE Transactions on Knowledge and Data Engineering. 38* (10), 6969–6986.

Zhong, Q., Wang, K., Xu, Z., Ding, L., Liu, J., & Du, B. (2024a). Achieving >97% on GSM8k: Deeply understanding the problems makes LLMs better solvers for math word problems. arXiv:2404.14963.

Zhong, W., Cui, R., Guo, Y., Liang, Y., Lu, S., Wang, Y., Saied, A., Chen, W., & Duan, N. (2024b). AGIEval: A human-centric benchmark for evaluating foundation models. In *Findings of the association for computational linguistics: NAACL 2024*.

Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., & Levy, O. (2023). LIMA: Less is more for alignment. In *Advances in neural information processing systems* (pp. 55006–55021). (*vol. 36*).